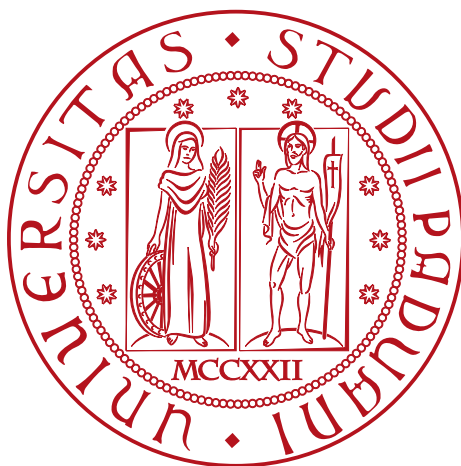


Università degli studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



**Servizio event-driven per l'estrazione
e la sintesi di KPI aziendali tramite
semantic layer e LLM**

Tesi di laurea triennale

Relatore

Prof. Federico Corò

Laureando

Giovanni Ponso

Matricola 2000558

«Frase ad effetto»

— Autore della suddetta frase

Ringraziamenti

Da scrivere al termine della stesura.

Padova, Settembre 2026

Giovanni Ponso

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di trecentoventi ore, dal laureando Giovanni Ponso presso l'azienda Datasoil S.r.l.. Lo stage è stato condotto sotto la supervisione del tutor aziendale Pietro De Caro, mentre il Prof. Federico Corò ha ricoperto il ruolo di tutor accademico.

Questa tesi tratta la progettazione e lo sviluppo di un servizio per la generazione automatica di riepiloghi testuali all'interno della piattaforma SaaS di facility management di Datasoil. Il servizio reagisce agli eventi della piattaforma, recupera in modo deterministico i KPI dell'entità interessata tramite un semantic layer e ne affida l'esposizione in linguaggio naturale a un modello linguistico (LLM), con l'obiettivo di supportare l'operatore sul campo con un quadro sintetico e attendibile dello stato di asset, ispezioni e ticket.

Indice

1	Introduzione	1.
1.1	L'azienda	1.
1.2	Il progetto e lo stage	2.
1.2.1	Descrizione	2.
1.2.2	Principali problematiche e relative soluzioni	2.
1.2.3	Pianificazione	4.
1.2.4	Svolgimento effettivo	5.
1.2.5	Obiettivi	6.
1.3	Il prodotto finale	7.
1.4	Organizzazione del testo	7.
2	Strumenti e tecnologie utilizzate	10.
2.1	Go	10.
2.2	MongoDB	10.
2.3	Cube	11.
2.4	PostgreSQL	11.
2.5	Modelli linguistici e API OpenAI	11.
2.6	Amazon SQS, SNS ed ElasticMQ	12.
2.7	Docker	12.
2.8	Strumenti per i test	13.
2.9	Librerie di supporto	13.
3	Analisi dei requisiti	14.
3.1	Casi d'uso	14.
3.1.1	Attori	15.
3.1.2	Interazioni previste	15.
3.1.2.1	UC1: Consultazione della sintesi	16.
3.1.2.2	UC2: Richiesta di produzione della sintesi	16.
3.1.3	Variabilità del contenuto	17.
3.2	Classificazione dei requisiti	17.
3.2.1	Requisiti funzionali	18.
3.2.2	Requisiti qualitativi	19.
3.2.3	Requisiti di vincolo	21.

3.3	Tracciamento dei requisiti	22.
3.3.1	Riepilogo quantitativo	22.
3.3.2	Tracciamento fonte-requisito	22.
3.3.3	Tracciamento caso d'uso-requisito	23.
4	Progettazione e sviluppo	26.
4.1	Studio preliminare degli approcci	26.
4.1.1	Il vincolo del determinismo	26.
4.1.2	Approcci valutati per i dati strutturati	27.
4.1.3	Interfacciamento tra la sorgente dati e il semantic layer	29.
4.1.4	Il recupero da documenti non strutturati	30.
4.2	Il semantic layer	31.
4.2.1	La sorgente relazionale	31.
4.2.2	Dimensioni e metriche	32.
4.2.3	La specializzazione per cliente	32.
4.2.4	La segregazione dei dati fra clienti	33.
4.2.5	Fin dove è arrivata la configurazione	35.
4.3	Scelte di progettazione	35.
4.3.1	La divisione in tre operazioni	35.
4.3.2	Comandi asincroni e letture sincrone	36.
4.3.3	Una coda per ciascun comando	37.
4.3.4	Il riepilogo assente	37.
4.3.5	Granularità dell'invalidazione	38.
4.3.6	Idempotenza	39.
4.3.7	Robustezza degli ingressi	39.
4.4	Architettura e struttura del codice	40.
4.4.1	Stile architetturale	40.
4.4.2	Organizzazione del codice	41.
4.4.3	Design pattern adottati	42.
4.5	Sviluppo dei componenti	44.
4.5.1	summary — Il coordinatore della produzione	45.
4.5.2	kpi — L'estrazione dei dati	49.
4.5.3	store — L'archivio dei riepiloghi	51.
4.5.4	tenant — Le impostazioni del cliente	52.
4.5.5	clients — Gli adattatori verso i sistemi esterni	53.
4.5.6	main.go — Il punto di ingresso e i consumatori	55.
4.5.7	http.go — La consultazione	57.
5	Verifica e validazione	60.
5.1	Approccio adottato	60.
5.2	Test automatici	61.

5.3 Test di sistema manuali	64.
5.4 Validazione del risultato	65.
5.5 Grado di soddisfacimento dei requisiti	65.
5.5.1 Tracciamento delle verifiche	66.
5.5.2 Approfondimento: la lunghezza del riepilogo	68.
5.6 Limiti della verifica svolta	69.
6 Conclusioni	72.
6.1 Raggiungimento degli obiettivi	72.
6.2 Sviluppi futuri	73.
6.3 Conoscenze acquisite	74.
6.4 Valutazione personale	75.
Glossario	76.
Bibliografia	78.

Elenco delle Figure

Figura 1 La funzionalità dal punto di vista dell'operatore.	15.
Figura 2 Diagramma dei casi d'uso del servizio.	16.
Figura 3 Architettura del servizio e flusso delle tre operazioni.	36.
Figura 4 I package del servizio e le dipendenze tra essi.	41.
Figura 5 Il flusso di produzione di un riepilogo.	44.
Figura 6 Le interfacce del package summary e i tipi che le realizzano.	46.

Elenco delle Tabelle

Tabella 1	Ripartizione delle ore prevista dal piano di lavoro.	5.
Tabella 2	Requisiti funzionali.	18.
Tabella 3	Requisiti qualitativi.	19.
Tabella 4	Requisiti di vincolo.	21.
Tabella 5	Riepilogo dei requisiti per tipologia e priorità.	22.
Tabella 6	Tracciamento tra fonti e requisiti.	22.
Tabella 7	Tracciamento tra casi d'uso e requisiti.	23.
Tabella 8	Confronto tra gli approcci valutati per il recupero dei dati strutturati.	28.
Tabella 9	Test automatici e requisiti verificati.	62.
Tabella 10	Copertura delle istruzioni per package.	63.
Tabella 11	Grado di soddisfacimento dei requisiti.	65.
Tabella 12	Modo di verifica di ciascun requisito.	67.
Tabella 13	Lunghezza in caratteri di sette generazioni per ciascun limite dichiarato nell'istruzione.	69.
Tabella 14	Raggiungimento degli obiettivi del piano di lavoro.	72.

Elenco dei Codici Sorgente

Capitolo 1

Introduzione

In questo capitolo presento l'azienda ospitante e introduco il progetto di stage: le problematiche affrontate, la pianificazione, gli obiettivi e il prodotto realizzato.

1.1 L'azienda

Datasoil S.r.l. è una software house con sede a Padova, parte del gruppo internazionale eFM. Attraverso la propria piattaforma SaaS B2B, l'azienda porta intelligenza artificiale ed *execution* mobile al centro delle *operations*, collegando mondo fisico e digitale in un unico layer operativo: dalle ispezioni alla manutenzione, fino alla gestione degli asset, le attività frammentate vengono trasformate in processi strutturati, scalabili e governabili. Datasoil opera nei settori *real estate*, industriale e manifatturiero, supportando organizzazioni complesse nell'evoluzione verso modelli operativi più integrati, predittivi e orientati al valore.

Il progetto di stage si colloca nell'ambito del *facility management*, ovvero la gestione integrata di edifici, impianti e asset aziendali, dove la piattaforma ruota attorno a tre entità fondamentali: gli **asset** (i beni gestiti, come impianti e attrezzature), le **ispezioni** (le verifiche periodiche condotte sugli asset) e i **ticket** (le segnalazioni e gli interventi di manutenzione).

Il prodotto è **multi-tenant**: una singola installazione serve più aziende clienti, i cui dati sono mantenuti rigorosamente separati. Come si vedrà nel corso del documento, questa caratteristica ha attraversato l'intero progetto di stage: dall'organizzazione dei dati, alla definizione delle metriche per i singoli clienti, fino alla configurazione delle interrogazioni per ciascun tenant.

1.2 Il progetto e lo stage

1.2.1 Descrizione

Il progetto di stage nasce dall'esigenza, individuata nel piano di lavoro, di efficientare l'operatività dell'utente sul campo: restituire in modo automatico una sintesi dello stato di manutenzione e dei relativi interventi per le diverse tipologie di entità gestite dalla piattaforma.

In concreto, si tratta di sviluppare un servizio che, all'apertura della scheda di un asset (o di un'ispezione o di un ticket), generi automaticamente un **riepilogo testuale in linguaggio naturale** dello stato dell'oggetto, sfruttando i dati aziendali per garantire l'attendibilità del contenuto e un modello linguistico (LLM) per la sua esposizione.

Il servizio è il solo oggetto dello stage. L'interfaccia con cui l'operatore visualizza il riepilogo e ne richiede la produzione appartiene alla piattaforma ed è competenza del team aziendale: nel documento viene descritta soltanto per dare senso al comportamento del servizio, che è stato progettato e realizzato per sostenerla.

Lo stage si è svolto in modalità full-time per un totale di 320 ore, sotto la supervisione del tutor aziendale Pietro De Caro. Gli obiettivi erano fissati dal piano di lavoro concordato con l'azienda, che mi ha lasciato però ampia autonomia sul *come* raggiungerli: per ciascun obiettivo ho valutato approcci alternativi, sviluppato prototipi per verificarne la fattibilità e consolidato le scelte attraverso frequenti momenti di confronto con il tutor.

La forma che il servizio ha infine assunto non era stabilita in partenza: è il risultato di scelte maturate lungo tutto il periodo, che la sezione seguente ripercorre insieme ai problemi da cui sono nate.

1.2.2 Principali problematiche e relative soluzioni

Le principali problematiche riscontrate durante la realizzazione del progetto riguardano l'affidabilità dei contenuti generati dall'LLM e l'interfacciamento con i dati aziendali. Di seguito vengono analizzati i problemi specifici e le relative soluzioni adottate.

1. Recupero deterministico dei dati per l'LLM

Descrizione: gli LLM eccellono nella produzione di testo fluente, ma se incaricati anche del recupero o del calcolo dei dati tendono a produrre *allucinazioni*, ovvero affermazioni plausibili ma non supportate dai dati reali. Gli approcci più diffusi per interrogare dati aziendali tramite LLM, ovvero il RAG e le tecniche Text-to-SQL, si sono rivelati dallo studio preliminare inadatti ai dati strutturati: in un contesto in cui il riepilogo supporta decisioni operative, un valore inventato non è accettabile.

Soluzione: è stato introdotto un *semantic layer* (Cube), uno strato intermedio che centralizza la definizione delle metriche e fornisce KPI «certificati», recuperati in modo completamente deterministico. All'LLM viene affidata soltanto l'esposizione in linguaggio naturale di valori già calcolati, con istruzioni che ne vincolano il comportamento ai soli dati forniti.

2. Interfacciamento tra il semantic layer e il database aziendale

Descrizione: la piattaforma conserva i dati su MongoDB, un database NoSQL, mentre Cube opera su sorgenti relazionali. L'azienda disponeva di una soluzione temporanea, applicata a un solo cliente: il travaso dei dati in un campo `jsonb` su PostgreSQL, poi «srotolato» in una vista materializzata. Nel tentativo di rendere l'architettura più generale ho sperimentato l'interrogazione diretta del campo `jsonb` grezzo da parte di Cube, ottenendo però tempi di risposta proibitivi e configurazioni SQL molto difficili da mantenere; ho poi sperimentato con esito positivo l'esportazione dei dati in formato Parquet, che però per reggersi nel tempo avrebbe richiesto un lavoro di automazione a carico dell'azienda. Questa problematica ha assorbito una parte significativa del tempo di stage.

Soluzione: in accordo con il tutor aziendale, l'ottimizzazione dell'interfacciamento è stata messa in pausa per non bloccare l'avanzamento del progetto, trattandosi di una questione prettamente infrastrutturale. Lo sviluppo è proseguito sull'ambiente PostgreSQL già pronto e configurato, mantenendo l'architettura aperta a una futura generalizzazione.

3. Gestione della multitenancy

Descrizione: i database dei vari clienti hanno schemi per lo più simili, ma presentano campi personalizzati; occorre inoltre garantire che ogni cliente possa accedere esclusivamente ai propri dati.

Soluzione: per il data model è stata utilizzata la direttiva `extends` di Cube, che permette di condividere le definizioni comuni estendendole in modo granulare per ciascun cliente; la segregazione dei dati si fonda invece sul fatto che a ciascun cliente corrispondono un database, un modello compilato e una cache propri, selezionati a partire dall'identificativo che il servizio presenta in un token JWT firmato. Anche le interrogazioni per il recupero dei KPI sono configurate per singolo tenant su MongoDB, senza interventi sul codice del servizio.

4. Definizione del perimetro dei KPI

Descrizione: una volta risolta la questione del recupero, restava da definire quali KPI fossero utili al riepilogo e in quale forma esporli: se sviluppare vere e proprie funzioni di analisi del dato (ad esempio il calcolo di trend), o se arricchire i KPI con dati qualitativi recuperati direttamente dal database.

Soluzione: confrontandomi con il tutor aziendale ho scelto di mantenere il sistema semplice: KPI aggregati «grezzi», affidando all’LLM la loro descrizione in linguaggio naturale guidata da istruzioni specifiche per ciascun blocco di dati. Il riepilogo accosta così due tipi di contenuto: i KPI aggregati e alcune informazioni puntuali sulle singole occorrenze, come la descrizione dei ticket aperti e dell’intervento svolto su quelli chiusi di recente. Anche queste ultime passano però dal semantic layer: l’ipotesi di recuperarle direttamente dal database, scavalcandolo, è stata scartata per non aprire una seconda via di accesso ai dati con garanzie diverse dalla prima.

5. Conservazione e aggiornamento del riepilogo

Descrizione: produrre un riepilogo richiede una chiamata al modello linguistico, quindi comporta un costo e alcuni secondi di attesa. Da qui la domanda su cosa farne una volta prodotto. Rigenerarlo a ogni apertura di scheda garantisce che sia sempre aggiornato, ma fa pagare quel costo anche all’operatore che apre la scheda per altri motivi. Conservarlo rende invece immediate le consultazioni successive, esponendo però al rischio opposto: quando i dati dell’entità cambiano, per esempio all’apertura di un nuovo ticket o al completamento di un’ispezione, il testo conservato smette di corrispondere alla realtà.

Soluzione: il riepilogo viene conservato, e utilizzo due meccanismi per evitare ri-generazioni inutili o versioni non valide del riepilogo.

Il primo separa la consultazione dalla produzione. La consultazione si limita a leggere: restituisce il riepilogo se esiste, altrimenti risponde che non è disponibile. Ricevendo questa seconda risposta la piattaforma può proporre all’operatore di richiedere la produzione, che resta l’unico modo per avviarla.

Il secondo tiene il riepilogo allineato ai dati. Ho valutato due strategie prima di scegliere: confrontare la data del riepilogo con quella dell’ultima modifica dell’entità, oppure assegnare al riepilogo una scadenza a tempo. Entrambe richiedevano un controllo periodico e lasciavano una finestra di tempo in cui il testo mostrato poteva essere già superato. La soluzione adottata parte invece da un meccanismo che la piattaforma aziendale possiede già, ovvero l’evento che segnala la modifica (update) di un’entità aziendale (ticket, ecc...). Quando il servizio intercetta una modifica, elimina subito dal database il riepilogo dell’entità di riferimento, così alla richiesta successiva sarà necessario generare un nuovo riepilogo basato su dati aggiornati.

1.2.3 Pianificazione

Il piano di lavoro concordato con l’azienda prevedeva otto settimane da quaranta ore ciascuna, per un totale di 320, così articolate:

1. introduzione al prodotto, alle tecnologie e ai processi aziendali; definizione degli obiettivi e avvio dell'analisi dei requisiti;
2. completamento dell'analisi dei requisiti; studio e confronto delle tecnologie e degli approcci RAG disponibili;
3. progettazione dell'applicativo e avvio della documentazione tecnica;
4. e 5. sviluppo dell'applicativo, a partire dai moduli principali;
6. completamento dello sviluppo funzionale ed esecuzione dei test unitari e di integrazione;
7. verifica dell'applicativo su dati reali e revisione della documentazione;
8. collaudo finale e rifinitura della documentazione.

La ripartizione delle ore prevista era la seguente:

Ore	Attività
8	Introduzione alla piattaforma proprietaria
8	Introduzione agli strumenti utilizzati in azienda
32	Analisi dei requisiti e stesura della relativa documentazione
32	Studio e confronto delle tecnologie disponibili
40	Progettazione dell'applicativo
120	Sviluppo dell'applicativo
40	Collaudo e test su dati reali
40	Scrittura della documentazione
320	Totale

Tabella 1: Ripartizione delle ore prevista dal piano di lavoro.

1.2.4 Svolgimento effettivo

Il lavoro ha seguito lo sviluppo previsto dal piano. Le differenze, concordate di volta in volta con il tutor aziendale, hanno riguardato soprattutto la distribuzione temporale di alcune fasi e l'esito dello studio preliminare. La fase di studio e confronto delle tecnologie, alla quale il piano destinava trentadue ore, ha prodotto il risultato più rilevante per l'intero progetto. Lo scopo dello stage era formulato attorno all'approccio RAG, ma

l'analisi ne ha evidenziato l'inadeguatezza rispetto ai dati strutturati, ai quali insieme al tutor aziendale è stata data la priorità. La distinzione tra dati strutturati e non strutturati era già presente negli obiettivi, i primi in O01 e O02, i secondi in O03 e O04. È divenuta così anche una distinzione di approccio: il RAG è rimasto la strada prevista per i soli documenti non strutturati.

A questa fase si è affiancata una questione che la pianificazione non contemplava: l'interfacciamento tra il semantic layer e il database aziendale. Trattandosi di un database non relazionale, il problema è di natura infrastrutturale e ha assorbito una porzione consistente del tempo.

L'analisi dei requisiti, che il piano collocava nelle prime due settimane, si è distribuita lungo l'intero periodo. L'autonomia lasciatami sul modo di realizzare gli obiettivi ha fatto sì che diversi requisiti si precisassero mentre il lavoro procedeva, restando a lungo impliciti nei confronti con il tutor; la loro formalizzazione, avvenuta nella parte conclusiva dello stage, ha reso evidenti alcuni punti su cui il servizio non era ancora allineato a quanto concordato, in particolare la lingua del testo e il fuso orario con cui vengono calcolate le date, il cui adeguamento è rientrato nell'ultima fase dello sviluppo.

Il collaudo previsto dalle settimane conclusive si è svolto nell'ambiente di sviluppo locale, sui dati reali dell'ambiente di *staging* aziendale, e si è accompagnato alla scrittura dei test automatici; l'integrazione nell'infrastruttura di produzione, corrispondente all'obiettivo desiderabile D01, non è stata invece realizzata.

1.2.5 Obiettivi

Il piano di lavoro concordato con l'azienda definisce i seguenti obiettivi obbligatori:

- **O01** — Ricezione di eventi asincroni dalle altre componenti della piattaforma, con caricamento delle configurazioni del cliente e delle entità interessate;
- **O02** — Valutazione di KPI specifici per entità e cliente, e interfacciamento con l'IA generativa per la produzione della sintesi testuale;
- **O03** — Indicizzazione delle entità per il recupero di documenti non strutturati (approccio RAG);
- **O04** — Ricezione di richieste utente in modalità agente, con recupero documentale e risposta.

A questi si aggiunge l'obiettivo desiderabile **D01**, l'integrazione nell'architettura e nel ciclo di rilascio della piattaforma.

Il lavoro si è concentrato sul flusso relativo ai dati strutturati (O01 e O02), portandolo a un livello di completezza e solidità vicino all'integrazione in produzione. Nella seconda metà dello stage, in accordo con il tutor aziendale, si è scelto di consolidare tale flusso anziché avviare

gli obiettivi O03 e O04: l'estensione ai documenti non strutturati avrebbe comportato lo studio e la realizzazione di un'infrastruttura di indicizzazione e recupero autonoma, con il rischio concreto di lasciare incompiuti entrambi i fronti. Gli obiettivi O03 e O04 sono stati pertanto analizzati sul piano teorico ma non realizzati, e sono ripresi tra gli sviluppi futuri nel Capitolo 6.

Anche l'obiettivo desiderabile D01 non è stato raggiunto: nella settimana conclusiva l'azienda ha preferito destinare il tempo residuo a un affiancamento formativo con uno sviluppatore del team, ritenendolo più utile rispetto a un'attività il cui coordinamento con il ciclo di rilascio interno avrebbe ecceduto la durata dello stage.

1.3 Il prodotto finale

Il servizio realizzato copre l'intero flusso previsto per i dati strutturati e offre tre operazioni distinte. La **produzione** di un riepilogo, richiesta tramite coda, recupera le interrogazioni configurate per il tenant, ne estrae i KPI tramite il semantic layer, genera il testo con l'LLM e lo rende persistente. La **consultazione**, servita in modo sincrono, restituisce il riepilogo di un'entità oppure segnala che non è disponibile, senza produrne di nuovi. È questa seconda risposta a permettere alla piattaforma di proporre all'operatore la produzione. L'**invalidazione**, attivata dall'evento con cui la piattaforma segnala la modifica di un'entità: elimina i riepiloghi che non ne rispecchiano più lo stato, così che la richiesta successiva ne produca di aggiornati.

Il servizio è configurabile per singolo cliente senza interventi sul codice, sia nelle interrogazioni che compongono il riepilogo, sia nella lingua e nel fuso orario con cui viene prodotto. È inoltre corredato di una suite di test automatici e di un ambiente di sviluppo locale che ne riproduce l'infrastruttura.

Per un'analisi approfondita del risultato ottenuto si rimanda al Capitolo 6.

1.4 Organizzazione del testo

La tesi è divisa in sei capitoli, che ripercorrono il lavoro nell'ordine in cui è stato svolto. Il primo, che si chiude con questa sezione, presenta l'azienda ospitante e il progetto di stage. Il Capitolo 2 descrive gli strumenti e le tecnologie utilizzate. Il Capitolo 3 delimita il sistema e ne analizza i requisiti. Il Capitolo 4 illustra lo studio degli approcci possibili, la progettazione e lo sviluppo del servizio. Il Capitolo 5 riporta le attività

di verifica e validazione. Il Capitolo 6 trae un bilancio del lavoro svolto e delle prospettive future.

Riguardo la stesura del testo sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini di uso non comune vengono definiti nel glossario, situato alla fine del documento, e alla loro prima occorrenza sono indicati con la nomenclatura *termini*;
- i termini in lingua straniera o facenti parte del gergo tecnico sono evidenziati con il carattere *corsivo*;
- i nomi di funzioni, variabili e altri elementi di codice sono scritti con carattere **monospaziato**;
- le citazioni a risorse presenti nella bibliografia sono affiancate dal rispettivo numero identificativo, es. [1].

Capitolo 2

Strumenti e tecnologie utilizzate

In questo capitolo presento le tecnologie utilizzate durante lo stage, motivandone per ciascuna il ruolo all'interno del progetto.

2.1 Go

Go è il linguaggio di programmazione utilizzato da Datasoil per lo sviluppo dei servizi backend, ed è il linguaggio in cui è stato scritto l'intero servizio oggetto dello stage. Nato in Google, Go privilegia la semplicità e la leggibilità del codice: offre una sintassi essenziale, una tipizzazione statica con tempi di compilazione rapidi, una libreria standard ricca e la compilazione in un singolo eseguibile, caratteristica che ne semplifica la distribuzione. Il linguaggio si distingue inoltre per il supporto nativo alla concorrenza, tramite le *goroutine* (thread leggeri gestiti dal runtime) e i canali di comunicazione tra esse.

Non avendo esperienza pregressa con il linguaggio, una parte dello stage è stata dedicata al suo apprendimento.

2.2 MongoDB

MongoDB è un database NoSQL orientato ai documenti: i dati sono memorizzati come documenti in formato BSON (una rappresentazione binaria di JSON) raggruppati in *collection*, senza uno schema rigido imposto a priori. È il database utilizzato da Datasoil per la persistenza dei dati, organizzati secondo il modello multi-tenant: un database logico per ciascun cliente.

Nel servizio sviluppato MongoDB svolge tre ruoli: contiene la configurazione delle interrogazioni per il recupero dei KPI, che ogni tenant può così definire senza modifiche al codice; custodisce i riepiloghi prodotti; ed è la sorgente da cui vengono lette le impostazioni del tenant che ne condizionano il contenuto, ovvero la lingua e il fuso orario.

2.3 Cube

Cube è un *semantic layer* open source: uno strato intermedio che si frappone tra le sorgenti dati e le applicazioni, centralizzando la definizione delle metriche di business. Invece di distribuire la logica di calcolo dei KPI tra le varie applicazioni, con il rischio di definizioni divergenti, le metriche vengono definite una sola volta nei *data model* di Cube, e le applicazioni le interrogano tramite API ottenendo valori consistenti e, in questo senso, «certificati».

Nel servizio sviluppato Cube è la sorgente di tutti i valori che compaiono nel riepilogo: le metriche sono definite nei suoi modelli dati e il servizio le interroga tramite API. Le ragioni per cui è stato scelto, e il modello dati costruito durante lo stage, sono discussi nel Capitolo 4.

2.4 PostgreSQL

PostgreSQL è il database relazionale su cui Cube esegue le proprie interrogazioni. Non è la sorgente originaria dei dati, che resta MongoDB, ma il punto in cui questi vengono resi disponibili in forma relazionale: i documenti sono travasati in una tabella e da lì esposti in una vista materializzata, le cui colonne sono ciò su cui i *data model* di Cube risultano definiti.

Il passaggio esiste perché Cube è progettato per operare su sorgenti relazionali, mentre la piattaforma conserva i propri dati in documenti. La soluzione era già predisposta dall'azienda per un cliente, e l'ho adottata per non fermare lo sviluppo su una questione infrastrutturale: le alternative valutate e le ragioni della scelta sono discusse nel Capitolo 4.

2.5 Modelli linguistici e API OpenAI

I modelli linguistici di grandi dimensioni (Large Language Model, LLM) sono modelli di apprendimento automatico addestrati su grandi quantità di testo, capaci di comprendere e produrre linguaggio naturale. Nel progetto l'LLM è impiegato con un ruolo deliberatamente circoscritto: non gli è richiesto di calcolare o dedurre alcun dato, ma soltanto di esporre in un testo scorrevole i KPI che gli vengono forniti, seguendo istruzioni che ne vincolano il comportamento ai soli dati presenti.

L'interazione avviene tramite le API di OpenAI, utilizzando l'SDK ufficiale per Go. Ogni generazione è una chiamata indipendente, composta da un messaggio di sistema, che definisce regole e vincoli espositivi, e da un messaggio utente contenente i dati, serializzati in JSON, corredati dalle istruzioni di lettura specifiche di ciascun blocco.

2.6 Amazon SQS, SNS ed ElasticMQ

Amazon Simple Queue Service (SQS) è il servizio di code gestite di AWS, utilizzato in Datasoil per la comunicazione asincrona tra le componenti della piattaforma. Una coda disaccoppia chi produce messaggi da chi li consuma: il produttore pubblica e prosegue, il consumatore elabora al proprio ritmo, e un messaggio la cui elaborazione fallisce torna disponibile per un nuovo tentativo.

Amazon Simple Notification Service (SNS) è il servizio di distribuzione di messaggi per argomenti dello stesso fornitore, e supplisce a un limite delle code: SQS consegna ciascun messaggio a un solo consumatore, quindi due servizi non possono ricevere lo stesso evento. Un messaggio pubblicato su un *topic* SNS viene invece recapitato in copia a tutti i sottoscrittori, tra i quali possono figurare altrettante code SQS. È il meccanismo che consente al servizio di ricevere gli eventi con cui la piattaforma segnala la modifica di un'entità, che hanno già altri destinatari, senza sottrarli a questi ultimi.

Il servizio sviluppato consuma due code distinte, una per ciascun comando che può ricevere: la richiesta di produzione di un riepilogo e la segnalazione che un'entità è stata modificata. Per il consumo si appoggia a un package sviluppato internamente dall'azienda, che si fa carico dell'intero dialogo con SQS: la ricezione dei messaggi tramite *long polling*, la loro elaborazione in parallelo da parte di un numero configurabile di *worker*, la cancellazione dalla coda dei soli messaggi elaborati con successo, così che quelli falliti vengano automaticamente riproposti, e l'arresto controllato del consumatore. Al package viene fornita unicamente la funzione di elaborazione del singolo messaggio, nella quale risiede la logica specifica del servizio.

Per lo sviluppo e i test in locale è stato utilizzato ElasticMQ, un server che espone un'API compatibile con SQS: il codice del servizio rimane identico a quello di produzione, cambia soltanto l'indirizzo della coda nella configurazione.

2.7 Docker

Docker è la piattaforma di containerizzazione utilizzata per riprodurre in locale l'infrastruttura necessaria allo sviluppo: l'istanza di Cube, il database MongoDB e la coda ElasticMQ vengono eseguiti come container, garantendo un ambiente di lavoro replicabile e vicino a quello di produzione.

Docker è impiegato anche dai test automatici, che se ne servono per avviare istanze temporanee di MongoDB secondo il meccanismo descritto nella sezione seguente.

2.8 Strumenti per i test

I test automatici del servizio sono scritti con il package `testing` della libreria standard di Go, senza librerie di asserzione esterne. È l'uso prevalente nell'ecosistema del linguaggio: la libreria standard si occupa dell'esecuzione, del confronto dei risultati e della misura della copertura, e un framework aggiuntivo introdurrebbe una dipendenza senza coprire un'esigenza rimasta scoperta.

A essa si affiancano due strumenti per i casi in cui il codice da verificare dialoga con l'esterno. Il package `net/http/httptest`, anch'esso nella libreria standard, esegue una richiesta HTTP direttamente sul gestore e ne raccoglie la risposta, senza bisogno di avviare un server né di occupare una porta di rete. La libreria `testcontainers` permette invece a un test di avviare un servizio reale dentro un container, utilizzarlo e distruggerlo al termine: nel progetto se ne serve per eseguire le interrogazioni su un'istanza vera di MongoDB, anziché su una sua imitazione che verificherebbe soltanto le chiamate e non il loro esito.

2.9 Librerie di supporto

Oltre alle tecnologie principali, il servizio si appoggia ad alcune librerie consolidate dell'ecosistema Go: `Viper` per la gestione della configurazione da file e variabili d'ambiente, `Resty` per le chiamate HTTP verso le API di Cube e la libreria ufficiale `mongo-driver` per l'accesso a MongoDB.

Il canale sincrono con cui la piattaforma consulta i riepiloghi è invece realizzato con il solo package `net/http` della libreria standard di Go, senza ricorrere a framework esterni: le esigenze del servizio si limitano a un singolo *endpoint* di lettura, e la libreria standard le copre integralmente.

Capitolo 3

Analisi dei requisiti

In questo capitolo delimito il confine del sistema, individuo gli attori e i casi d'uso, e da questi derivo i requisiti, classificati per tipologia e priorità e collegati alle rispettive fonti.

Il sistema oggetto dell'analisi è il servizio di contesto sviluppato durante lo stage. L'interfaccia con cui l'operatore visualizza la sintesi e ne richiede la produzione appartiene invece alla piattaforma, e resta perciò fuori dal confine considerato.

I requisiti raccolti in questo capitolo esprimono **che cosa** il sistema deve fare, cioè le capacità che un osservatore esterno può riscontrare, e non il modo in cui tali capacità vengono realizzate. Le soluzioni adottate sono decisioni di progettazione e trovano spazio nel Capitolo 4.

3.1 Casi d'uso

L'analisi parte da ciò che accade sulla piattaforma, perché è da lì che nascono le richieste rivolte al servizio. Dal punto di vista dell'operatore la funzionalità ha due soli esiti, illustrati in Figura 1: aprendo la scheda di un'entità, la sintesi è già disponibile, oppure è assente e se ne può richiedere la produzione. Da queste due possibilità nascono le interazioni descritte nel seguito.

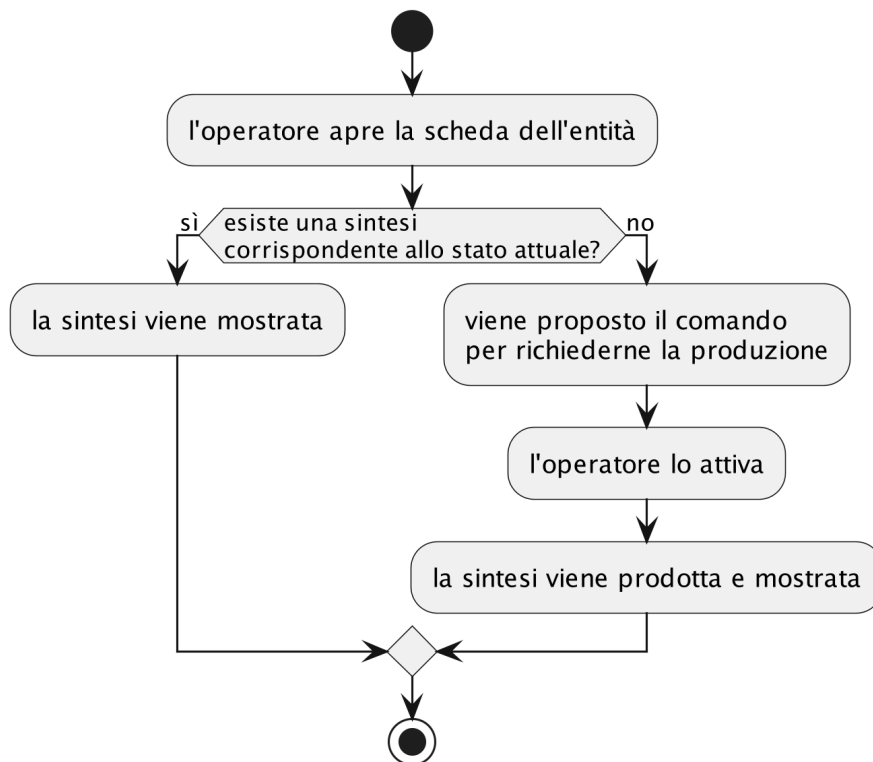


Figura 1: La funzionalità dal punto di vista dell'operatore.

3.1.1 Attori

L'operatore non contatta mai il servizio in modo diretto. Agisce sull'interfaccia della piattaforma, ed è la piattaforma a rivolgersi al servizio. L'unico attore individuato è quindi la **piattaforma SaaS**, che invia le richieste e ne riceve gli esiti.

L'operatore sul campo beneficia della funzionalità, ma non è un attore del sistema in esame perché non interagisce mai con il suo confine. È un portatore di interesse, e la sua esigenza è ciò che motiva l'intera analisi. Non sono stati individuati attori secondari. I sistemi da cui il servizio ottiene i dati e con cui produce il testo della sintesi non sono un presupposto dell'analisi: derivano dalle scelte architetturali illustrate nel Capitolo 4.

3.1.2 Interazioni previste

Tra la piattaforma e il servizio sono previste due interazioni, rappresentate in Figura 2 e descritte ciascuna da un caso d'uso. La prima è la

consultazione di una sintesi, la seconda è la **richiesta di produzione** di una sintesi non ancora disponibile.

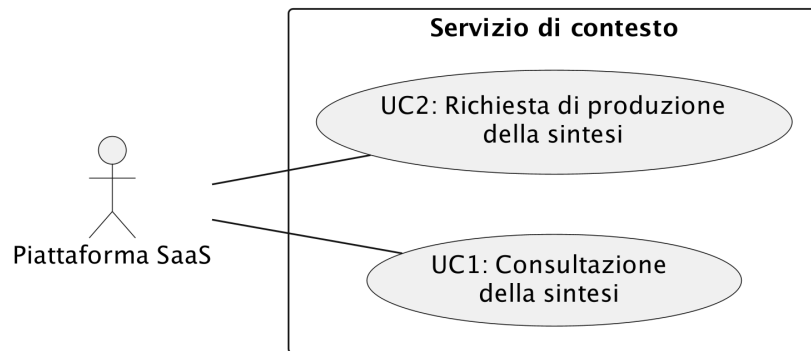


Figura 2: Diagramma dei casi d'uso del servizio.

3.1.2.1 UC1: Consultazione della sintesi

Attore principale Piattaforma SaaS

Scenario principale la piattaforma chiede al servizio la sintesi di una determinata entità, e il servizio la restituisce.

Precondizioni il servizio è operativo e la richiesta riguarda un'entità del tenant per cui la piattaforma è autorizzata.

Postcondizioni la piattaforma dispone di una sintesi che corrisponde allo stato attuale dell'entità.

Estensioni se per l'entità non è disponibile alcuna sintesi, il servizio lo segnala. La piattaforma può allora proporre all'operatore di richiederne la produzione (Sezione 3.1.2.2).

Trigger un operatore apre la scheda dell'entità sulla piattaforma.

3.1.2.2 UC2: Richiesta di produzione della sintesi

Attore principale Piattaforma SaaS

Scenario principale la piattaforma chiede la produzione della sintesi per una determinata entità e un determinato evento. Il servizio individua i dati previsti per quel caso, li recupera, ne produce una sintesi in linguaggio naturale e la rende disponibile alle consultazioni successive.

Precondizioni il servizio è operativo, e per quel tenant e quel tipo di evento è definito l'insieme dei dati da comporre.

Postcondizioni la sintesi dell'entità è disponibile alla consultazione.

Estensioni se una parte dei dati previsti non è disponibile, viene omessa dalla sintesi anziché comprometterne la produzione. Se la

produzione non va a buon fine non viene resa disponibile alcuna sintesi, e la richiesta viene ripresentata.

Trigger un operatore richiede la produzione dalla scheda dell'entità, oppure un evento della piattaforma la richiede automaticamente.

3.1.3 Variabilità del contenuto

I due casi d'uso non si moltiplicano per il tipo di entità: ispezioni, ticket e asset condividono lo stesso schema di interazione, e ciò che cambia da un caso all'altro sono soltanto i dati che compongono la sintesi.

Questi variano lungo due dimensioni. La prima è il tipo di entità, perché di un'ispezione e di un ticket interessano cose diverse. La seconda è il cliente: ogni tenant misura la propria operatività a modo proprio, dispone di dati differenti e attribuisce importanza a indicatori differenti, per cui la sintesi di un'ispezione non ha lo stesso contenuto presso clienti diversi.

Quali dati comporre non è quindi un requisito del sistema, ma una sua configurazione. Il requisito è che il sistema sappia differenziare il contenuto in base al tipo di entità e all'evento (RF-OB_03) e che tale differenziazione sia definibile per ciascun cliente senza modifiche al codice (RQA-DE_01); quali indicatori compaiano poi nella sintesi di un determinato cliente non appartiene a questo capitolo. Il meccanismo che rende possibile la configurazione è descritto nel Capitolo 4.

3.2 Classificazione dei requisiti

I requisiti individuati sono identificati da un codice così strutturato:

[TIPO]-[PRIORITÀ]_[NUMERO]

dove il **tipo** assume uno dei seguenti valori:

RF requisito funzionale, ovvero una capacità che il sistema deve offrire;

RQA requisito qualitativo, ovvero una proprietà che il comportamento del sistema deve rispettare;

RV requisito di vincolo, ovvero una condizione imposta dall'ambiente in cui il sistema si inserisce.

e la **priorità** uno tra:

OB obbligatorio, necessario al funzionamento della soluzione;

DE desiderabile, di valore riconoscibile ma non indispensabile;

OP opzionale, possibile sviluppo successivo.

Due requisiti funzionali compaiono tra i desiderabili e gli opzionali pur derivando da obiettivi che il piano di lavoro classificava come obbligatori, e conviene chiarirlo prima di leggere le tabelle. Si tratta di RF-DE_01 e RF-OP_01, corrispondenti agli obiettivi O03 e O04, relativi al recupero di informazioni da documenti non strutturati e all'interazione in modali-

tà agente. In accordo con il tutor aziendale il perimetro del lavoro è stato concentrato sui dati strutturati, per le ragioni esposte nel Capitolo 1, e i due requisiti sono stati ricollocati di conseguenza. Il Capitolo 6 li riprende tra gli sviluppi futuri.

Tra i requisiti di vincolo rientrano soltanto le condizioni che l'azienda ha effettivamente imposto, ovvero il linguaggio di sviluppo, la sorgente dati esistente e l'infrastruttura di comunicazione già adottata dalla piattaforma. Le altre tecnologie impiegate non compaiono qui perché sono state scelte nel corso del progetto: sono quindi decisioni di progettazione, e vengono discusse nel Capitolo 4.

3.2.1 Requisiti funzionali

Codice	Descrizione	Fonte
RF-OB_01	Il sistema deve produrre una sintesi testuale in linguaggio naturale dello stato di un'entità a partire dai suoi dati.	Piano di lavoro (O02)
RF-OB_02	Il sistema deve comporre la sintesi con i KPI aggregati dell'entità e con informazioni qualitative puntuali che la riguardano.	Riunione con il tutor aziendale
RF-OB_03	Il sistema deve differenziare il contenuto della sintesi in base al tipo di entità e all'evento che la richiede.	Piano di lavoro (O02)
RF-OB_04	Il sistema deve fornire la sintesi di un'entità quando richiesta, oppure indicare che non è disponibile.	Riunione con il tutor aziendale
RF-OB_05	Il sistema deve consentire di richiedere la produzione di una sintesi non ancora disponibile.	Riunione con il tutor aziendale

Codice	Descrizione	Fonte
RF-OB_06	Il sistema deve mantenere disponibile la sintesi prodotta per le richieste successive, senza riprodurla a ogni consultazione.	Riunione con il tutor aziendale
RF-OB_07	Il sistema deve impedire che venga fornita una sintesi non più corrispondente allo stato dell'entità.	Riunione con il tutor aziendale
RF-OB_08	Il sistema deve poter essere attivato dalle altre componenti della piattaforma, e non soltanto da un'azione dell'operatore.	Piano di lavoro (O01)
RF-DE_01	Il sistema deve indicizzare le entità per consentire il recupero di informazioni da documenti non strutturati.	Piano di lavoro (O03)
RF-OP_01	Il sistema deve rispondere a richieste dell'utente in modalità agente.	Piano di lavoro (O04)

Tabella 2: Requisiti funzionali.

3.2.2 Requisiti qualitativi

Codice	Descrizione	Fonte
RQA-OB_01	I valori esposti nella sintesi devono provenire esclusivamente dai dati recuperati, senza deduzioni né informazioni non presenti nei dati.	Piano di lavoro (O02)

Codice	Descrizione	Fonte
RQA-OB_02	La sintesi deve essere prodotta nella lingua predefinita del tenant.	Riunione con il tutor aziendale
RQA-OB_03	Le date riportate nella sintesi devono corrispondere a quelle che l'operatore vede sulla piattaforma.	Analisi interna
RQA-OB_04	Ogni tenant deve poter accedere esclusivamente ai propri dati.	Piano di lavoro (O01)
RQA-OB_05	Nessuna richiesta accettata deve andare perduta, neppure in caso di arresto del servizio, che deve portare a termine le elaborazioni già avviate.	Analisi interna
RQA-OB_06	Richieste non valide o errori temporanei delle sorgenti dati non devono compromettere il funzionamento del servizio.	Analisi interna
RQA-OB_07	L'elaborazione ripetuta della stessa richiesta non deve produrre effetti indesiderati.	Analisi interna
RQA-DE_01	I dati da comporre nella sintesi e le relative istruzioni di esposizione devono essere configurabili per singolo tenant senza modifiche al codice.	Analisi interna
RQA-DE_02	La sintesi deve potersi mantenere entro una lunghezza massima stabilita, scegliendo	Riunione con il tutor aziendale

Codice	Descrizione	Fonte
	do quali contenuti riportare in ordine di importanza quando i dati eccedono lo spazio disponibile.	

Tabella 3: Requisiti qualitativi.

Il requisito RQA-DE_02, sulla lunghezza della sintesi, compare tra i desiderabili e non tra gli obbligatori. La ragione è che nulla nel sistema dipende dal suo rispetto: il servizio funziona e la sintesi resta utilizzabile anche quando il testo eccede, e l'interfaccia che imporrebbe un vincolo di spazio non è ancora stata realizzata. Il suo valore è rivolto a uno scenario futuro, quello di entità con una storia molto lunga, dove i dati da riportare eccederebbero qualunque spazio disponibile e occorrerebbe scegliere quali riportare. Il Capitolo 5 ne riporta la misura e l'esito.

3.2.3 Requisiti di vincolo

Codice	Descrizione	Fonte
RV-OB_01	Il servizio deve essere sviluppato nel linguaggio Go.	Vincolo aziendale
RV-OB_02	Il servizio deve attingere alla sorgente dati esistente su MongoDB, organizzata con un database per ciascun tenant.	Vincolo aziendale
RV-OB_03	La comunicazione asincrona con la piattaforma deve avvenire tramite la coda Amazon SQS, impiegando la libreria di consumo sviluppata internamente dall'azienda.	Vincolo aziendale
RV-DE_01	Il servizio deve integrarsi nell'architettura e nel ciclo di rilascio della piattaforma.	Piano di lavoro (D01)

Tabella 4: Requisiti di vincolo.

3.3 Tracciamento dei requisiti

Le matrici seguenti collegano ciascun requisito alla propria origine e ai casi d'uso che lo hanno generato. Servono a due verifiche: che ogni requisito abbia una giustificazione, e che ogni caso d'uso trovi nei requisiti tutto ciò che serve a realizzarlo.

3.3.1 Riepilogo quantitativo

Tipologia	Obbligatori	Desiderabili	Opzionali	Totale
Funzionali (RF)	8	1	1	10
Qualitativi (RQA)	7	2	0	9
Di vincolo (RV)	3	1	0	4
Totale	18	4	1	23

Tabella 5: Riepilogo dei requisiti per tipologia e priorità.

3.3.2 Tracciamento fonte-requisito

Fonte	Requisiti generati
Piano di lavoro (O01)	RF-OB_08, RQA-OB_04
Piano di lavoro (O02)	RF-OB_01, RF-OB_03, RQA-OB_01
Piano di lavoro (O03)	RF-DE_01

Fonte	Requisiti generati
Piano di lavoro (O04)	RF-OP_01
Piano di lavoro (D01)	RV-DE_01
Riunioni con il tutor aziendale	RF-OB_02, RF-OB_04, RF-OB_05, RF-OB_06, RF-OB_07, RQA-OB_02, RQA-DE_02
Analisi interna	RQA-OB_03, RQA-OB_05, RQA-OB_06, RQA-OB_07, RQA-DE_01
Vincoli aziendali	RV-OB_01, RV-OB_02, RV-OB_03

Tabella 6: Tracciamento tra fonti e requisiti.

3.3.3 Tracciamento caso d'uso-requisito

Caso d'uso	Requisiti coinvolti
UC1: Consultazione	RF-OB_04, RF-OB_06, RF-OB_07, RQA-OB_04, RQA-OB_06
UC2: Richiesta di produzione	RF-OB_01, RF-OB_02, RF-OB_03, RF-OB_05, RF-OB_06, RF-OB_08, RQA-OB_01, RQA-OB_02, RQA-OB_03, RQA-OB_04, RQA-OB_05, RQA-OB_06, RQA-OB_07, RQA-DE_01, RQA-DE_02

Tabella 7: Tracciamento tra casi d'uso e requisiti.

I requisiti RF-DE_01 e RF-OP_01 non compaiono nella matrice dei casi d'uso. Derivano direttamente dal piano di lavoro e riguardano capacità

che il perimetro dello stage non comprende: i relativi casi d'uso andranno definiti nel momento in cui verranno affrontate.

Capitolo 4

Progettazione e sviluppo

In questo capitolo espongo lo studio degli approcci possibili, il modello dati con cui i KPI sono definiti, le scelte di progettazione che hanno dato forma al servizio, la struttura che ne è derivata e lo sviluppo dei singoli componenti, documentato sul codice effettivo.

Il Capitolo 3 ha stabilito che cosa il sistema deve fare. Questo capitolo espone il come, ed è diviso in due metà. La prima riguarda i dati: lo studio che ha portato a scegliere il modo in cui arrivano al modello linguistico, e il modello dati costruito per definirli. La seconda riguarda il servizio: le decisioni che ne hanno determinato la forma, la struttura che ne è risultata e infine i componenti, presentati uno a uno insieme al codice che li realizza.

4.1 Studio preliminare degli approcci

4.1.1 Il vincolo del determinismo

Il servizio deve produrre un testo che descrive lo stato di un'entità, e quel testo viene letto da un operatore che sulla sua base prende decisioni di manutenzione. Tra i clienti della piattaforma figurano organizzazioni che gestiscono infrastrutture critiche, dove un valore riportato in modo errato non è un difetto estetico ma un rischio operativo.

Il vincolo che ne deriva è il punto di partenza di tutta la progettazione: nessun valore esposto nel riepilogo può essere inventato, dedotto o ricalcolato. È il requisito RQA-OB_01, e da solo determina il ruolo che il modello linguistico può assumere.

I modelli linguistici sono strumenti probabilistici: producono il testo più plausibile date le istruzioni ricevute, non il testo verificato rispetto a una fonte. Quando la plausibilità coincide con la verità il risultato è corretto; quando non coincide il modello produce comunque un testo scorrevole e sicuro di sé, fenomeno noto come *allucinazione*. La conseguenza pratica

è che a un modello linguistico si può affidare l'esposizione di un dato, ma non il suo recupero né il suo calcolo.

Ho quindi separato nettamente le due responsabilità. Il recupero e il calcolo dei valori avvengono per via deterministica, fuori dal modello; al modello arriva un insieme di valori già stabiliti, con il compito di comporli in un testo leggibile e con il divieto esplicito di aggiungere interpretazioni soggettive. Tutto lo studio che segue riguarda il modo di realizzare la prima metà di questa separazione.

4.1.2 Approcci valutati per i dati strutturati

I KPI di un'entità risiedono nel database della piattaforma, in forma strutturata. Ho valutato tre modi di portarli davanti al modello.

Text-to-SQL: Il modello linguistico riceve lo schema del database e la domanda dell'utente, e produce l'interrogazione da eseguire. È l'approccio più flessibile, perché non richiede di prevedere in anticipo quali domande verranno poste. Ha però un difetto insanabile rispetto al vincolo posto sopra: l'interrogazione stessa è generata in modo probabilistico. Un'interrogazione sintatticamente valida ma semanticamente sbagliata, per esempio una che aggrega sul campo sbagliato o dimentica un filtro, restituisce un risultato numericamente plausibile e formalmente ineccepibile, e nulla nel testo finale segnala che sia sbagliato. Lo spostamento della generazione dal testo all'interrogazione non elimina il problema, lo rende soltanto più difficile da individuare.

Calcolo dei KPI affidato al modello: Il modello riceve i dati grezzi e ne calcola direttamente gli aggregati. È l'approccio più diretto da realizzare, ed è anche quello che viola nel modo più evidente il vincolo: il calcolo è esattamente l'operazione che un modello linguistico non garantisce. Su volumi di dati non banali si aggiunge il problema della finestra di contesto, che impone di ridurre i dati prima di passarli, introducendo una seconda fonte di imprecisione (oltre che costi altissimi).

Semantic layer: Le metriche sono definite una sola volta in un modello dati dichiarativo, esterno sia al database sia alle applicazioni, e vengono interrogate attraverso un'API. La definizione di «numero di ispezioni non conformi negli ultimi dodici mesi» esiste in un punto solo, è scritta da chi conosce il dominio ed è la stessa per tutte le applicazioni che la richiedono [1]. Il modello linguistico non partecipa in alcun modo al calcolo: riceve il risultato di un'interrogazione predeterminata.

La Tabella 8 riassume il confronto.

Criterio	Text-to-SQL	Calcolo dal modello	Semantic layer
Determinismo del valore	Assente: la query è generata dal modello	Assente: il calcolo è svolto dal modello	Garantito: query predefinite
Verificabilità di un errore	Bassa: il risultato è plausibile anche se sbagliato	Bassa: nessun riscontro sul valore prodotto	Alta: la metrica è ispezionabile e riproducibile
Latenza	Due chiamate al modello per ogni richiesta	Una chiamata su un volume di dati elevato	Interrogazioni dirette, con caching nativo
Manutenibilità	La correttezza dipende dallo schema esposto al modello	Nessuna definizione condivisa delle metriche	Definizione unica e centralizzata
Multi-tenancy	Da realizzare a parte	Da realizzare a parte	Nativa, con estensione dei modelli e token JWT

Tabella 8: Confronto tra gli approcci valutati per il recupero dei dati strutturati.

La scelta è caduta sul semantic layer, e il prodotto adottato è Cube. Oltre a soddisfare il vincolo di determinismo, offre già realizzate alcune funzionalità che altrimenti sarebbero state da sviluppare: il caching delle interrogazioni, le API di accesso e un sistema di sicurezza multi-tenant basato su token JWT. Lo sviluppo di un backend dedicato avrebbe richiesto di riprodurle tutte, senza alcun vantaggio compensativo. L'accostamento fra semantic layer e modello linguistico non è isolato. Lyft, che ha costruito uno strato analogo per le stesse ragioni, ne espone oggi le definizioni agli agenti di intelligenza artificiale proprio perché la

loro struttura deterministica riduce le allucinazioni nelle analisi generate [1].

Va osservato che la scelta ha un costo, ed è opportuno dichiararlo: il semantic layer non risponde a domande impreviste. Ogni metrica che non è stata definita non esiste, e il servizio non può recuperarla. Questo è accettabile perché il riepilogo non è una conversazione ma un testo a struttura nota, il cui contenuto è deciso in anticipo dal tipo di entità e dall'evento che lo richiede. Sarebbe inaccettabile in uno scenario di interrogazione libera, che è infatti l'obiettivo O04 rimasto fuori dal perimetro. Va detto che anche in quello scenario il semantic layer resterebbe utile: far generare al modello un'interrogazione fra le metriche e le dimensioni dichiarate è un problema molto più circoscritto che fargli scrivere SQL libero.

4.1.3 Interfacciamento tra la sorgente dati e il semantic layer

La scelta del semantic layer ha aperto una questione che il piano di lavoro non prevedeva. La piattaforma conserva i dati su MongoDB, un database orientato ai documenti, mentre Cube è progettato per operare su sorgenti relazionali. Serviva quindi un modo per rendere i dati di MongoDB interrogabili da Cube, e questa è la parte dello stage che ha assorbito più tempo rispetto alle attese.

Ho valutato tre strade.

Vista materializzata su PostgreSQL. I documenti vengono travasati in un campo `jsonb` su PostgreSQL e da lì «srotolati» in una vista materializzata con le colonne che servono. È la soluzione che l'azienda aveva già predisposto per un singolo cliente. Ha il pregio di presentare a Cube una tabella relazionale ordinaria, e il difetto di richiedere una vista scritta a mano per ciascun cliente.

Interrogazione diretta del campo `jsonb`. Saltare la vista e lasciare che sia Cube a estrarre i campi dal documento grezzo, così da eliminare il passaggio da mantenere per ogni cliente. L'ho sperimentata perché avrebbe reso l'architettura generale, ma i tempi di risposta si sono rivelati proibitivi e le configurazioni SQL necessarie difficili da leggere e da mantenere.

Esportazione in Parquet interrogata da DuckDB. Concettualmente la soluzione migliore tra quelle esaminate: DuckDB è già incluso in Cube, quindi non aggiunge componenti all'infrastruttura, e legge i file Parquet in modo efficiente senza richiedere una vista per cliente. L'ho sperimentata e ha funzionato. A fermarla non è stato un limite tecnico, ma il passaggio successivo: per diventare la soluzione stabile avrebbe richiesto di automatizzare l'esportazione periodica dei dati da MongoDB, un lavoro infrastrutturale che l'azienda non era nelle condizioni di affrontare in quel momento.

La questione è infatti di natura infrastrutturale e riguarda il modo in cui l'azienda espone i propri dati, non il servizio oggetto dello stage: qualunque delle tre soluzioni si adotti, il servizio interroga Cube nello stesso identico modo. Constatato che il tempo speso stava sottraendosi allo sviluppo, in accordo con il tutor aziendale ho proseguito sull'ambiente PostgreSQL già configurato, lasciando la generalizzazione come intervento successivo.

A posteriori osservo che la rinuncia è stata più netta del necessario. L'automazione serviva all'esercizio continuativo in produzione, dove i dati cambiano di continuo; il lavoro dello stage si è invece svolto interamente in ambiente di sviluppo, su dati che non avevano bisogno di essere aggiornati a ogni istante, e per quello sarebbe bastata un'esportazione manuale eseguita una volta. Avrei potuto proseguire su Parquet e DuckDB senza dipendere da alcuna decisione infrastrutturale, lasciando all'azienda il solo passo dell'automazione. Averlo capito dopo è il tipo di errore che si commette quando si confonde ciò che serve al prodotto con ciò che serve al proprio lavoro.

4.1.4 Il recupero da documenti non strutturati

L'obiettivo O03 del piano di lavoro prevedeva di estendere il riepilogo alle informazioni contenute in documenti non strutturati, come procedure e manuali, attraverso un approccio RAG (*Retrieval-Augmented Generation*): i documenti vengono suddivisi in frammenti e indicizzati, e al momento della richiesta i frammenti più pertinenti vengono recuperati e forniti al modello insieme alla domanda.

Lo studio ha portato a individuare come soluzione appropriata un recupero ibrido, che combina la ricerca vettoriale per somiglianza semantica con la ricerca lessicale BM25, seguito da una fase di *re-ranking* dei risultati [2]. La ricerca vettoriale da sola tende a mancare le corrispondenze esatte su codici e sigle, che in un contesto di manutenzione sono frequenti; la ricerca lessicale da sola non coglie le riformulazioni.

L'approccio è stato analizzato ma non realizzato. La ragione non è tecnica ma di perimetro: richiede un'infrastruttura di indicizzazione e di recupero autonoma, il cui sviluppo avrebbe occupato la parte restante dello stage lasciando incompiuto il flusso sui dati strutturati. La decisione, concordata con il tutor aziendale, è discussa nel Capitolo 1 e ripresa tra gli sviluppi futuri.

Vale però osservare che la separazione di responsabilità stabilita all'inizio di questo capitolo rimane valida anche in quello scenario. Un frammento di documento recuperato è un dato reale, non una deduzione del modello, e viene fornito al modello nello stesso modo in cui gli vengono forniti i KPI. La struttura del servizio, come si vedrà nella Sezione 4.5.1, è

predisposta ad accogliere questa seconda sorgente senza modifiche al flusso.

4.2 Il semantic layer

Scelto il semantic layer, restava da costruirlo. I KPI da cui nasce un riepilogo non esistono finché qualcuno non li definisce, e definirli ha occupato una parte consistente dello stage: vivono in un modello dati dichiarativo che Cube compila e traduce in SQL al momento dell'interrogazione. Questa sezione descrive quel modello, che insieme al servizio è l'altra metà di ciò che ho realizzato.

4.2.1 La sorgente relazionale

Cube interroga PostgreSQL, dove i documenti di MongoDB arrivano in due forme: una tabella di appoggio conserva ciascun documento intero in un campo **jsonb**, mentre una vista materializzata ne espone in colonne i campi di uso più frequente. Ogni cubo del modello parte da una **SELECT** che unisce le due, così da disporre sia delle colonne già pronte sia del documento originale per tutto ciò che la vista non espone.

Il secondo caso non è raro, perché alcune informazioni vivono in profondità dentro il documento. Un ticket, per esempio, non ha un campo con la data di chiusura: ha un elenco di transizioni di stato, e la chiusura è la prima transizione il cui tipo è **closed**.

```
1 # cube/model/shared/tickets_base.yml
2 - name: closed_at
3   sql: >
4     (SELECT (t.value ->> 'ts')::timestampz
5      FROM jsonb_array_elements(
6        ({CUBE}.raw -> 'status') -> 'transitions'
7      ) t(value)
8      WHERE t.value ->> 'metatype' = 'closed'
9      ORDER BY (t.value ->> 'ts')::timestampz
10     LIMIT 1)
11   type: time
```

È il genere di espressione che, moltiplicata per ogni campo, rendeva difficile da mantenere l'interrogazione diretta del **jsonb** discussa nella Sezione 4.1.3. Scritta nel modello dati viene però scritta una volta sola, e tutto ciò che ne dipende si limita a nominarla.

4.2.2 Dimensioni e metriche

Un cubo dichiara dimensioni e metriche. Le dimensioni sono gli attributi su cui si può filtrare e raggruppare; le metriche sono i valori aggregati che si possono chiedere.

```
1  measures:
2    - name: total_tickets
3      type: count
4
5    - name: open_tickets
6      type: count
7      filters:
8        - sql: "{closed_at} IS NULL"
9
10   - name: avg_resolution_days
11     sql: >
12       EXTRACT(EPOCH FROM
13         ({closed_at} - {opened_at})) / 86400
14     type: avg
15     filters:
16       - sql: "{closed_at} IS NOT NULL"
```

Nell'esempio il valore di questa forma sta nella riga **filters** di **open_tickets**, e vale per qualunque metrica definita allo stesso modo. Che cosa significhi «ticket aperto» è stabilito in un punto solo, ed è un ticket privo di transizione di chiusura. Chiunque chieda **open_tickets**, il servizio di contesto oggi o una dashboard domani, riceve lo stesso numero calcolato nello stesso modo. È la ragione per cui il semantic layer è stato scelto, e qui si vede messa in pratica.

Si noti anche che le definizioni si appoggiano l'una all'altra: **closed_at** è estratta dal documento, **open_tickets** la usa come filtro e **avg_resolution_days** la usa come estremo di un intervallo. La complessità dell'estrazione resta confinata in un punto e tutto il resto ne discende.

4.2.3 La specializzazione per cliente

Le definizioni comuni stanno in **model/shared**, quelle di un singolo cliente in **model/tenants/<identificativo>**. Un cubo del cliente estende quello condiviso, tramite la direttiva **extends** [3], e vi aggiunge ciò che serve soltanto a lui.

```
1  # cube/model/tenants/6902.../inspections_ferrovie.yml
2  cubes:
3    - name: inspection_ferrovie
4      extends: inspections_base
```

```

5
6     dimensions:
7       - name: physical_risk
8         sql: >
9           CASE upper({CUBE}.risk_physical)
10            WHEN 'BASSO' THEN 0
11            WHEN 'MEDIO' THEN 1
12            WHEN 'ALTO' THEN 2 END
13         type: number
14
15     measures:
16       - name: physical_risk_max
17         sql: "{physical_risk}"
18         type: max

```

Questo cliente registra i livelli di rischio come testo, e per poterne calcolare il peggiore occorre prima tradurli in numeri. È una necessità sua: un altro cliente potrebbe non avere affatto i livelli di rischio, oppure misurarli su una scala diversa. È la variabilità per cliente descritta nel Capitolo 3, vista dal lato della realizzazione.

Le interrogazioni che il servizio invia a Cube non sono scritte nel codice ma configurate per ciascun cliente, secondo il meccanismo descritto nella Sezione 4.5.2. L'estensione spiega perché quelle configurate per questo cliente nominano `inspection_ferrovie` e non `inspections_base`, pur usando membri che appartengono al cubo condiviso.

```

1  {
2    "measures": [
3      "inspection_ferrovie.physical_risk_max",
4      "inspection_ferrovie.total_completed_inspections"
5    ],
6    "timeDimensions": [{
7      "dimension": "inspection_ferrovie.inspection_date",
8      "granularity": "year"
9    }]
10 }

```

`physical_risk_max` è definita nel file del cliente, mentre `total_completed_inspections` e `inspection_date` vengono dal file condiviso: dopo l'estensione appartengono tutte allo stesso cubo.

4.2.4 La segregazione dei dati fra clienti

A ogni interrogazione il servizio allega un token JWT che porta l'identificativo del tenant, firmato dal client descritto nella Sezione 4.5.5. Cube lo legge dal contesto di sicurezza e, se manca, rifiuta la richiesta.

```

1 // cube/cube.js
2 function getTenantId(context) {
3   const fromToken = context && context.securityContext
4     && context.securityContext.tenant_id;
5   if (fromToken) {
6     return fromToken;
7   }
8   throw new Error('Missing tenant_id in JWT token');
9 }

```

Da quell'identificativo discendono tre decisioni. La prima riguarda quali definizioni caricare: ai file condivisi si aggiungono quelli della cartella del cliente, e il modello compilato risulta diverso da cliente a cliente.

```

1 repositoryFactory: function (context) {
2   const tenantId = getTenantId(context);
3   return {
4     dataSchemaFiles: async function () {
5       const shared = loadDir(
6         path.join(__dirname, 'model', 'shared'));
7       const tenantFiles = loadDir(
8         path.join(__dirname, 'model', 'tenants', tenantId));
9       return shared.concat(tenantFiles);
10    },
11  };
12 },

```

La seconda riguarda il database a cui connettersi, che è quello intestato al cliente.

```

1 driverFactory: function (context) {
2   const tenantId = getTenantId(context);
3   return {
4     type: 'postgres',
5     host: process.env.CUBEJS_DB_HOST,
6     user: process.env.CUBEJS_DB_USER,
7     password: process.env.CUBEJS_DB_PASSWORD,
8     database: tenantId,
9   };
10 }

```

La terza riguarda ciò che Cube conserva fra un'interrogazione e l'altra. Per non ricompilare il modello e non ricalcolare i risultati a ogni richiesta, Cube tiene in memoria lo schema compilato e gli esiti delle interrogazioni già eseguite. Se queste memorie fossero comuni a tutti, una richiesta di un cliente potrebbe ricevere un risultato calcolato sui dati di un altro. Le due funzioni `contextToAppId` e `contextToOrchestratorId` lo impediscono assegnando a ciascun cliente le proprie, etichettate con il suo identificativo.

Ne segue che la segregazione richiesta da RQA-OB_04 non è un filtro applicato alle interrogazioni, ma la conseguenza del fatto che due clienti non condividono nulla: né le definizioni, né la connessione, né la cache. Un'interrogazione scritta male non può raggiungere i dati di un altro cliente, perché il database in cui quei dati vivono non è nemmeno aperto. È una garanzia più solida di quella offerta da un filtro, che dipenderebbe invece dalla correttezza di chi lo scrive.

4.2.5 Fin dove è arrivata la configurazione

Il modello costruito durante lo stage copre un solo cliente e due dei tre tipi di entità della piattaforma. Per le ispezioni sono definiti l'andamento dei livelli di rischio negli anni, le non conformità rilevate e le statistiche sulle ispezioni svolte; per i ticket le statistiche complessive, l'elenco di quelli aperti e di quelli chiusi di recente con la descrizione dell'intervento eseguito. Gli asset, terzo tipo di entità, non sono stati configurati.

Va sottolineato che queste metriche valgono per quel cliente e non per il sistema: un altro tenant, disponendo di dati diversi e misurando la propria operatività a modo suo, avrebbe un modello diverso a parità di codice del servizio.

4.3 Scelte di progettazione

Le decisioni raccolte in questa sezione non riguardano un singolo componente ma la forma complessiva del servizio. Sono le scelte su cui mi sono confrontato più a lungo con il tutor aziendale, e per ciascuna riporto le alternative valutate e la ragione della preferenza. La prima riguarda quali operazioni il servizio debba offrire, e da essa discendono tutte le altre.

4.3.1 La divisione in tre operazioni

Il servizio poteva esporre una sola operazione, quella che produce il riepilogo quando qualcuno lo chiede. Ne espone invece tre, che corrispondono a tre momenti distinti nella vita di un riepilogo.

La **produzione** costruisce un riepilogo. Ricevuti il tenant, l'entità e l'evento che l'ha richiesta, recupera le interrogazioni configurate per quel caso, ne ottiene i valori dal semantic layer, li affida al modello linguistico e conserva il testo ottenuto.

La **consultazione** restituisce un riepilogo già prodotto, oppure segnala che non ne esiste alcuno. Non produce nulla e non ha effetti: è una sola lettura.

L'**invalidazione** elimina i riepiloghi di un'entità che è stata modificata, così che smettano di essere restituiti dalla consultazione.

Il fatto che la consultazione non produca mai nulla è la conseguenza più importante di questa divisione, e le sottosezioni che seguono ne discutono le implicazioni una per una. La Figura 3 mostra il flusso completo.

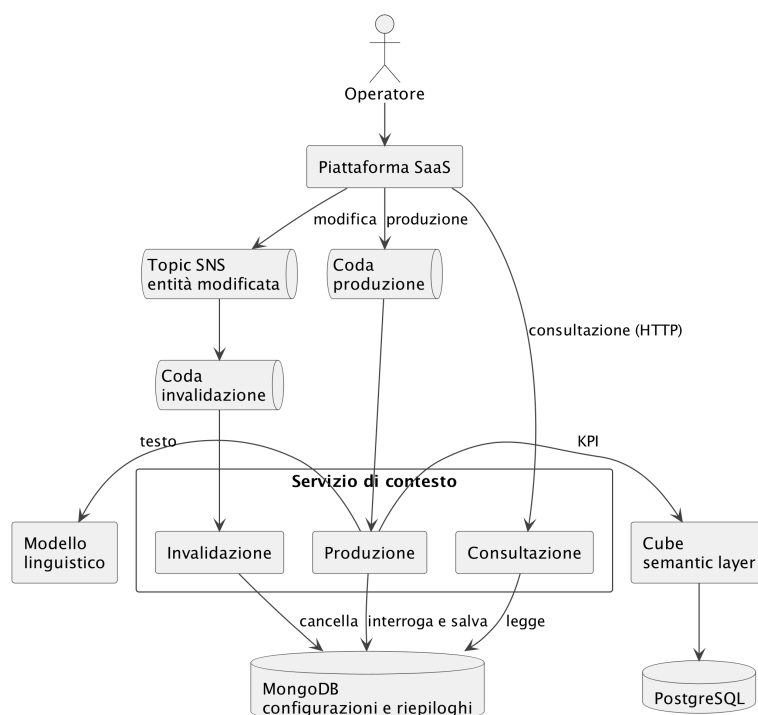


Figura 3: Architettura del servizio e flusso delle tre operazioni.

4.3.2 Comandi asincroni e letture sincrone

La prima versione del servizio riceveva ogni cosa dalla coda. Era una scelta comoda in prototipazione, ma inadeguata alla consultazione: l'operatore apre una scheda e attende una risposta, mentre una coda non risponde a chi le scrive.

Ho quindi separato i due canali secondo la natura delle operazioni. La produzione e l'invalidazione sono comandi: modificano lo stato, richiedono tempo, possono fallire e in tal caso vanno ritentati. Una coda offre esattamente questo, perché conserva il messaggio finché non viene elaborato con successo e assorbe i picchi di richieste senza sovraccaricare il servizio. La consultazione è invece una lettura: è veloce, non ha effetti, e chi la esegue ha bisogno della risposta immediatamente. Un endpoint HTTP sincrono risponde a questa esigenza.

Resta da spiegare come la piattaforma venga a sapere che una produzione richiesta è terminata. Il servizio non la avvisa: SQS non offre al mittente alcun riscontro sull'esito dell'elaborazione, e costruire un canale di notifica dedicato avrebbe aggiunto una componente al solo scopo di comunicare un fatto già osservabile. L'interfaccia interroga periodicamente la consultazione finché il riepilogo non compare, con un limite di tempo oltre il quale segnala all'operatore che la richiesta non è andata a buon fine. Il meccanismo riusa un'operazione che esiste già ed è la più economica del servizio.

4.3.3 Una coda per ciascun comando

I due comandi potevano viaggiare su una sola coda, distinti da un campo del messaggio. Ho preferito due code separate.

I due messaggi hanno origini e forme diverse. La richiesta di produzione è definita insieme al servizio e ne segue il formato; la segnalazione di modifica appartiene alla piattaforma, esisteva già per altri scopi e arriva incapsulata nella struttura del sistema di distribuzione. Con una coda sola il consumatore dovrebbe riconoscere il formato prima di poterlo leggere, ramificare sul tipo di comando e cercare l'identificativo del tenant in posizioni diverse a seconda dei casi: una logica di smistamento che non appartiene a nessuno dei due flussi e che va mantenuta insieme a entrambi.

Anche i volumi e i costi sono asimmetrici. Un'invalidazione è una cancellazione, arriva a ogni modifica di un'entità e può presentarsi a ondate quando la piattaforma esegue aggiornamenti massivi. Una produzione comporta diverse interrogazioni e una chiamata al modello linguistico, e dietro di essa c'è un operatore che aspetta. Su una coda condivisa un'ondata di invalidazioni si metterebbe in fila davanti alla richiesta di quell'operatore; con due code i due flussi hanno consumatori distinti e non si ostacolano.

Il costo infrastrutturale non entra nel confronto perché SQS si tariffa sul numero di richieste e non sul numero di code: due code che smaltiscono lo stesso traffico di una costano uguale.

4.3.4 Il riepilogo assente

Il riepilogo prodotto viene conservato su MongoDB, e questo pone una domanda: che cosa deve fare la consultazione quando il riepilogo richiesto non esiste.

La risposta abituale per un valore conservato è quella della cache, cioè calcolarlo al momento e restituirlo a chi lo ha chiesto. Applicata qui, significherebbe che aprire la scheda di un'entità priva di riepilogo ne avvia la produzione: l'operatore attende alcuni secondi senza averlo

chiesto, e ogni apertura di scheda comporta una chiamata al modello, che è l'operazione più costosa del servizio. Verrebbe inoltre meno la ragione della richiesta esplicita di produzione, perché la consultazione avrebbe già prodotto.

Il servizio adotta quindi il comportamento opposto: se il riepilogo non c'è, lo dichiara e si ferma. Sta alla piattaforma decidere cosa farne, e la scelta naturale è proporre all'operatore di richiederne la produzione. Il costo viene sostenuto solo quando qualcuno lo ha voluto.

Questa scelta è il motivo per cui la lettura dell'archivio non appartiene al package che produce i riepiloghi ma è raggiunta direttamente dall'endpoint di consultazione, come si vedrà nella Sezione 4.5.7. Se la consultazione passasse per il codice di produzione, la tentazione di far produrre in caso di assenza sarebbe a una riga di distanza. Tenendo separati i due percorsi la garanzia è strutturale e non affidata alla disciplina di chi scriverà il codice in seguito.

4.3.5 Granularità dell'invalidazione

Un'entità può avere più riepiloghi, uno per ciascun evento che li ha richiesti. Quando arriva la segnalazione che l'entità è cambiata, il servizio li cancella tutti.

La segnalazione dice che l'entità è stata modificata e nulla più: non conosce l'esistenza dei riepiloghi né quali dati ciascuno di essi utilizzi. L'alternativa era dichiarare, per ogni riepilogo, da quali fonti dipende, e cancellare solo quelli effettivamente toccati. L'ho scartata perché la sua correttezza si reggerebbe su una dichiarazione mantenuta a mano: chi aggiungesse una fonte a un'interrogazione dimenticando di aggiornare la dichiarazione non provocherebbe alcun guasto visibile, e semplicemente un giorno un operatore leggerebbe un dato vecchio senza avere modo di accorgersene.

Il confronto è quindi tra due errori. Cancellare più del necessario comporta la riproduzione di un riepilogo che era ancora valido, e l'errore si esaurisce lì. Cancellare meno del necessario lascia in circolazione un testo non aggiornato che viene mostrato come se fosse attuale, e questo l'operatore non ha modo di rilevarlo.

Il primo errore costa inoltre poco. La cancellazione di per sé non riproduce nulla: il costo si presenta solo se qualcuno apre davvero quella scheda e richiede il riepilogo, e in quel caso lo ottiene aggiornato.

Va segnalato un vincolo infrastrutturale che questa scelta comporta. SQS è punto a punto: un messaggio consegnato a un consumatore non è più disponibile per gli altri. Il servizio non può quindi mettersi in ascolto sulla coda con cui la piattaforma già distribuisce gli eventi di modifica, perché sottrarrebbe i messaggi al loro destinatario originario. Serve invece iscrivere una coda dedicata al topic SNS da cui quegli eventi

provengono, così che il sistema di distribuzione ne recapiti una copia a ciascun consumatore.

4.3.6 Idempotenza

Elaborare due volte lo stesso messaggio non deve produrre effetti diversi dall'elaborarlo una volta sola. Non è una precauzione teorica ma la conseguenza diretta di come funziona l'infrastruttura scelta: SQS garantisce la consegna almeno una volta, il che significa che lo stesso messaggio può essere recapitato due volte anche in assenza di errori. Vi si aggiunge il fatto che la libreria di consumo interna non verifica l'esito della cancellazione del messaggio dalla coda: se quella cancellazione fallisce, il messaggio ricompare e viene rielaborato.

Entrambi i comandi sono quindi costruiti perché la ripetizione sia innocua, come richiede RQA-OB_07. La produzione controlla per prima cosa se il riepilogo esiste già e in tal caso lo restituisce senza rifarlo, evitando una seconda chiamata al modello. L'invalidazione cancella ciò che trova e considera un esito valido il non aver trovato nulla, perché l'entità poteva legittimamente non avere riepiloghi. Il codice corrispondente è mostrato nelle sezioni 4.5.1 e 4.5.3.

Si noti che il controllo iniziale della produzione ha un effetto collaterale voluto: una richiesta di produzione su un riepilogo esistente non lo rigenera. Per ottenere un riepilogo aggiornato occorre prima invalidare, che è esattamente ciò che accade quando i dati cambiano davvero.

4.3.7 Robustezza degli ingressi

Il servizio riceve richieste da tre canali e da sistemi che non controlla. Un messaggio malformato, un identificativo assente o un'indisponibilità temporanea del semantic layer non devono comprometterne il funzionamento, come prescrive RQA-OB_06.

Ogni richiesta viene quindi validata prima di essere elaborata: quelle prive del tenant, dell'identificativo dell'entità o dell'evento sono rifiutate subito, senza attivare il flusso e senza spendere una chiamata al modello. Il servizio prosegue nel frattempo con i messaggi successivi, e un ingresso scorretto resta un fatto locale a quel messaggio.

Su questo punto il servizio presenta una lacuna. Un messaggio rifiutato non viene cancellato dalla coda, perché la cancellazione avviene solo in caso di successo, e torna quindi disponibile per un nuovo tentativo. Trattandosi di un messaggio malformato il tentativo fallirà di nuovo, e il messaggio verrà ripresentato indefinitamente. Il rimedio previsto da SQS è la coda di scarto (*dead letter queue*), sulla quale i messaggi vengono spostati dopo un numero configurato di tentativi falliti, in modo da

togliarli dal ciclo e conservarli per l'ispezione. Non è stata configurata nel corso dello stage, e la riprendo tra gli sviluppi futuri nel Capitolo 6. Un criterio diverso vale all'interno della produzione, sui singoli blocchi di dati. Un'interrogazione che restituisce un risultato vuoto non fa fallire nulla: il blocco arriva al modello privo di valori, e le istruzioni gli impongono di omettere ciò che manca invece di inventarlo. Se invece è il semantic layer a non rispondere, il fallimento riguarda la richiesta nel suo complesso e non un suo frammento: la produzione si interrompe, nessun riepilogo parziale viene conservato e il messaggio torna sulla coda per essere ritentato quando il sistema sarà di nuovo raggiungibile.

4.4 Architettura e struttura del codice

Le decisioni appena esposte hanno dato al servizio una forma precisa. Questa sezione la descrive su tre piani: lo stile con cui il servizio si inserisce nella piattaforma e si organizza al proprio interno, la disposizione dei package che ne discende, e i pattern di progettazione con cui quello stile diventa codice.

4.4.1 Stile architetturale

Il servizio non viene invocato: resta in ascolto e reagisce a ciò che gli altri componenti della piattaforma producono. L'integrazione avviene per **scambio di messaggi asincroni**, e i due canali di ingresso portano messaggi di natura diversa. Sulla coda di produzione arrivano comandi, cioè richieste rivolte al servizio perché compia un'operazione. Sul canale di invalidazione arrivano notifiche di eventi: la piattaforma dichiara che un'entità è stata modificata, senza stabilire che cosa se ne debba fare, e la distribuzione avviene in *publish-subscribe* attraverso un topic al quale più consumatori possono iscriversi [4]. È questa seconda forma a rendere il servizio autonomo nel mantenere validi i propri riepiloghi. Alle due code si affianca il canale sincrono della consultazione, riservato alle letture.

All'interno, il servizio segue lo stile **ports and adapters**, noto anche come architettura esagonale [5]. Il nucleo applicativo è il package **summary**, che non conosce alcun sistema esterno: dichiara le interfacce di ciò che gli occorre, e quelle interfacce sono le porte. Le realizzazioni concrete stanno fuori e ne sono gli adattatori: l'archivio su MongoDB, il servizio che recupera i KPI, il client del modello linguistico, il lettore delle impostazioni del cliente. Lo stesso criterio si ripete un livello più sotto, perché anche il servizio dei KPI dichiara a sua volta l'interfaccia del semantic layer di cui ha bisogno. Dal lato opposto stanno gli adattatori in ingresso, cioè i due consumatori di coda e il gestore dell'endpoint HTTP, la funzione che riceve la richiesta di consultazione e vi risponde. A

collegare le due sponde è il *composition root*, l'unico luogo del programma che conosce insieme le porte e gli adattatori.

Ciò che tiene insieme questa struttura è la direzione delle dipendenze, che puntano tutte verso il nucleo: è la persistenza a dipendere dall'interfaccia dichiarata dal dominio, non il dominio a dipendere da MongoDB. Da qui discendono due proprietà concrete. Sostituire l'archivio o il fornitore del modello linguistico non tocca la logica di produzione, perché il nucleo continua a vedere la stessa porta. E nei test quelle stesse porte accolgono realizzazioni finte, così il flusso si esercita per intero senza rete e senza costi.

Da questo schema il servizio si discosta in un punto, deliberatamente: la consultazione non attraversa il nucleo, perché quel gestore legge l'archivio direttamente. È la scelta che le impedisce di produrre riepiloghi, per le ragioni esposte nella Sezione 4.3.

4.4.2 Organizzazione del codice

Il servizio è scritto in Go e segue l'organizzazione convenzionale del linguaggio, che distingue il codice eseguibile dai package di libreria. Sotto `cmd` risiede il punto di ingresso, che legge la configurazione, apre le connessioni verso i sistemi esterni, costruisce i componenti e avvia i tre ascoltatori. Sotto `internal` risiedono i package applicativi, ciascuno con una responsabilità circoscritta.

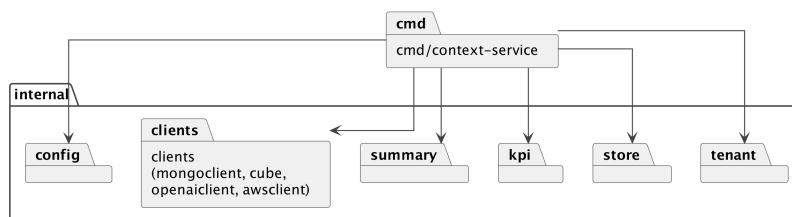


Figura 4: I package del servizio e le dipendenze tra essi.

summary coordina la produzione di un riepilogo, dalla raccolta dei dati alla chiamata al modello;

kpi recupera le interrogazioni configurate per il tenant e l'evento, le esegue sul semantic layer e ne restituisce i risultati;

store conserva, restituisce e cancella i riepiloghi prodotti;

tenant legge le impostazioni del cliente che influenzano il contenuto del riepilogo;

clients contiene gli adattatori verso i sistemi esterni, ovvero MongoDB, Cube, OpenAI e AWS;

config raccoglie e valida la configurazione da file e variabili d'ambiente.

Come mostra la Figura 4, le frecce delle dipendenze puntano tutte dall'alto verso il basso: il punto di ingresso conosce ogni package, mentre i package applicativi non si conoscono tra loro e nessuno di essi importa gli altri. Il criterio che rende possibile questa struttura è che le dipendenze sono dichiarate da chi le usa e non da chi le fornisce, ed è approfondito nella Sezione 4.5.1.

4.4.3 Design pattern adottati

Lo stile descritto sopra si realizza nel codice attraverso tre pattern di progettazione [6]. Tutti e tre applicano lo stesso principio, programmare verso un'interfaccia anziché verso una realizzazione, ma rispondono a domande distinte.

Dependency injection. La domanda è chi decida quale realizzazione un componente userà. Il coordinatore della produzione ha bisogno di un archivio, di un modello linguistico, di una sorgente di dati e delle impostazioni del cliente: se se li costruisse da sé dovrebbe conoscere MongoDB, OpenAI e Cube, e non sarebbe verificabile senza di essi. Li riceve invece dall'esterno al momento della costruzione, sotto forma di quattro valori che soddisfano le interfacce dichiarate.

```
1 // internal/summary/summary.go
2 func New(store Store, llm LLM, kpi Source,
3     tenants Tenants) *Service {
4     return &Service{store: store, llm: llm,
5         kpi: kpi, tenants: tenants}
6 }
```



La scelta delle realizzazioni concrete avviene in un punto solo dell'intero programma, il composition root.

```
1 // cmd/context-service/main.go
2 summaries := store.New(db)
3 tenants := tenant.New(db)
4 kpiService := kpi.New(db, cubeClient)
5 summaryService := summary.New(
6     summaries, llm, kpiService, tenants)
```



Il guadagno immediato è la verificabilità: nei test quelle stesse quattro posizioni ricevono realizzazioni finte, e il flusso di produzione si esercita per intero senza rete e senza costi. Il pattern realizza inoltre il principio di inversione delle dipendenze, perché il nucleo dipende da astrazioni che possiede e non da tipi concreti che gli sono esterni.

Repository. La domanda è quale parte del sistema debba sapere in che modo i dati sono conservati. Il coordinatore deve poter salvare un riepilogo e rileggerlo, ma non ha ragione di sapere che quel riepilogo vive in una collection di MongoDB individuata da una coppia di campi.

L'interfaccia è quindi espressa nei termini del dominio, l'entità e l'evento, e non in quelli del database.

```
1 // internal/summary/summary.go
2 type Store interface {
3     Get(ctx context.Context, tenantID, geocID,
4         trigger string) (string, bool, error)
5     Save(ctx context.Context, tenantID, geocID,
6         trigger, summary string) error
7 }
```

Tutto ciò che è specifico di MongoDB, ovvero i filtri, l'*upsert*, l'indice unico e i limiti di tempo, resta confinato nel package `store` descritto nella Sezione 4.5.3. Se l'archivio cambiasse tecnologia, il coordinatore non se ne accorgerebbe.

Adapter. La domanda è come far dialogare due interfacce concepite per scopi diversi. Al servizio serve una sola operazione, con due stringhe in ingresso e una in uscita. L'SDK di OpenAI ne espone un'altra: richiede una struttura di parametri con il modello e la temperatura, un elenco tipizzato di messaggi distinti per ruolo, e restituisce un insieme di risposte alternative fra cui scegliere. Le due forme sono incompatibili, e il package `openaiclient` è il punto in cui una viene tradotta nell'altra.

```
1 // quello che serve al servizio
2 // internal/summary/summary.go
3 type LLM interface {
4     Complete(ctx context.Context, systemPrompt,
5         userMessage string) (string, error)
6 }

1 // la traduzione verso l'SDK
2 // internal/clients/openaiclient/openaiclient.go
3 func (c *Client) Complete(ctx context.Context,
4     systemPrompt, userMessage string) (string, error) {
5     ...
6     resp, err := c.api.Chat.Completions.New(ctx,
7         openai.ChatCompletionNewParams{
8             Model:      openai.ChatModel(c.model),
9             Temperature: openai.Float(0.2),
10            Messages: []openai.ChatCompletionMessageParamUnion{
11                openai.SystemMessage(systemPrompt),
12                openai.UserMessage(userMessage),
13            },
14        })
15     ...
16     return resp.Choices[0].Message.Content, nil
17 }
```

Le due stringhe che il servizio passa diventano i due messaggi che l'SDK richiede, e la prima delle risposte alternative torna indietro come stringa. Il vocabolario dell'SDK non compare in nessun altro file del servizio: cambiare fornitore significa scrivere un secondo adattatore, non modificare la logica di produzione. Lo stesso vale per i client verso MongoDB, Cube e AWS.

4.5 Sviluppo dei componenti

Resta da vedere il codice. Le sottosezioni che seguono attraversano i package nell'ordine in cui il flusso di produzione li incontra, e mostrano come le decisioni della Sezione 4.3 siano state realizzate.

La Figura 5 segue una richiesta di produzione dal messaggio in coda fino al riepilogo conservato, e mostra in quale ordine i componenti entrano in gioco. Il riquadro contrassegnato da **alt** racchiude due decorsi alternativi, quello in cui il riepilogo esiste già e quello in cui va prodotto; il riquadro **loop** indica un passo ripetuto.

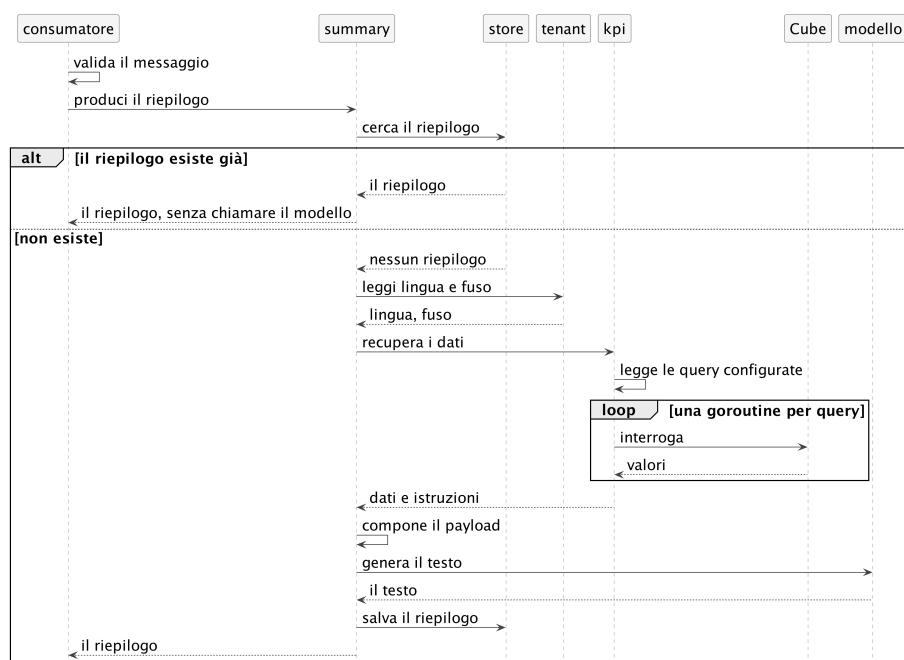


Figura 5: Il flusso di produzione di un riepilogo.

Il consumatore valida il messaggio e chiama il coordinatore, che per prima cosa interroga l'archivio: se il riepilogo esiste già lo restituisce e il flusso finisce lì, senza spendere nulla. Altrimenti legge lingua e fuso del cliente, chiede i dati al servizio dei KPI, che esegue in parallelo le

interrogazioni configurate per quell'evento, riunisce i risultati in un unico payload, lo affida al modello linguistico e conserva il testo ottenuto.

Le sezioni che seguono percorrono lo stesso tragitto e mostrano il codice con cui le decisioni della Sezione 4.3 sono state realizzate: sono quindi un secondo passaggio sulle stesse scelte, questa volta dal lato dell'implementazione. Gli estratti provengono dai file effettivi del servizio; alcune righe sono spezzate per adattarle alla pagina e le porzioni non rilevanti sono sostituite da puntini.

4.5.1 summary — Il coordinatore della produzione

Il package `summary` contiene la logica centrale del servizio: a partire da un tenant, un'entità e un evento, produce il riepilogo. Per farlo ha bisogno di quattro collaborazioni esterne: una sorgente di dati, un modello linguistico, un archivio in cui salvare e le impostazioni del cliente. La decisione strutturale più importante del package è che nessuna di queste collaborazioni è un riferimento concreto: sono quattro interfacce, dichiarate dal package stesso.

```
1 // internal/summary/summary.go
2
3 type Source interface {
4     Blocks(ctx context.Context, tenantID, geocID,
5         trigger, timezone string) (map[string]any, error)
6 }
7
8 type LLM interface {
9     Complete(ctx context.Context, systemPrompt,
10         userMessage string) (string, error)
11 }
12
13 type Store interface {
14     Get(ctx context.Context, tenantID, geocID,
15         trigger string) (string, bool, error)
16     Save(ctx context.Context, tenantID, geocID,
17         trigger, summary string) error
18 }
19
20 type Tenants interface {
21     Get(ctx context.Context,
22         tenantID string) (language, timezone string, err error)
23 }
24
25 type Service struct {
26     store Store
```

```

27  llm      LLM
28  kpi      Source
29  tenants  Tenants
30  }

```

In Go le interfacce sono soddisfatte implicitamente: un tipo le realizza per il solo fatto di possedere i metodi richiesti, senza dichiararlo. La convenzione del linguaggio è che l'interfaccia appartenga a chi la consuma e non a chi la fornisce, ed è l'inverso di quanto avviene nei linguaggi a oggetti tradizionali. La conseguenza è visibile nella Figura 6: i package che realizzano le quattro interfacce non importano **summary**, e **summary** non importa loro. Le frecce di realizzazione attraversano i confini dei package senza che alcun file di codice le dichiari.

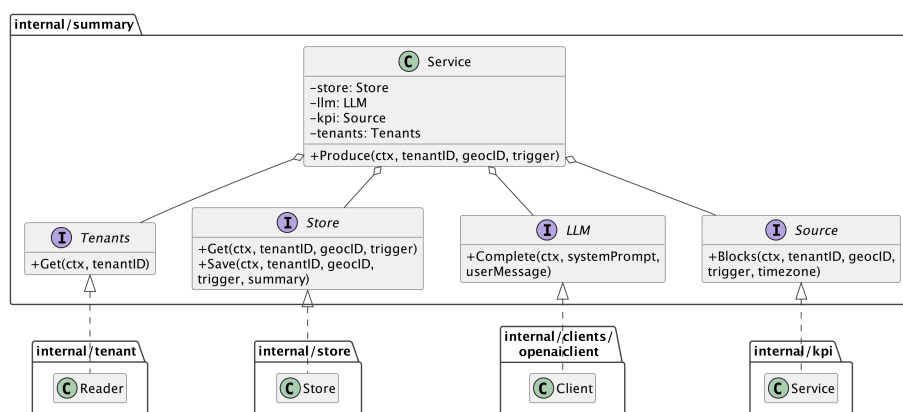


Figura 6: Le interfacce del package summary e i tipi che le realizzano.

Questa impostazione ha due effetti pratici. Il primo è che **summary** non sa che i riepiloghi finiscono su MongoDB né che il modello è quello di OpenAI: sostituire l'uno o l'altro non tocca la logica di produzione. Il secondo riguarda i test, ed è quello che ha inciso di più: fornendo realizzazioni finte delle quattro interfacce, il flusso di produzione si verifica per intero senza chiamare il modello linguistico, quindi senza costi e in modo ripetibile, come descritto nel Capitolo 5.

Il metodo **Produce** percorre l'intero flusso, e lo riporto integralmente perché è il cuore del servizio. I commenti segnano i punti in cui una decisione della Sezione 4.3 diventa codice.

```

1  // internal/summary/summary.go
2
3  func (s *Service) Produce(ctx context.Context,
4      tenantID, geocID, trigger string) (string, error) {
5
6      // se la sintesi c'è già non la rifacciamo: sqs può
7      // consegnare due volte lo stesso messaggio (idempotenza)

```

```

8     summary, found, err := s.store.Get(ctx, tenantID,
9         geocID, trigger)
10    if err != nil {
11        return "", fmt.Errorf(
12            "reading stored summary: %w", err)
13    }
14    if found {
15        return summary, nil
16    }
17
18    language, timezone, err := s.tenants.Get(ctx, tenantID)
19    if err != nil {
20        return "", fmt.Errorf("tenant settings: %w", err)
21    }
22
23    // ogni sorgente produce i suoi blocchi; li fondiamo
24    // in un'unica mappa così l'llm viene chiamato una
25    // volta sola, con tutto il contesto davanti
26    data := map[string]any{}
27    for _, src := range s.sourcesFor(trigger) {
28        blocks, err := src.Blocks(ctx, tenantID, geocID,
29            trigger, timezone)
30        if err != nil {
31            return "", err
32        }
33        for name, block := range blocks {
34            data[name] = block
35        }
36    }
37
38    payload, err := json.MarshalIndent(
39        map[string]any{"geoc_id": geocID, "data": data},
40        "", " ")
41    if err != nil {
42        return "", fmt.Errorf("building payload: %w", err)
43    }
44
45    summary, err = s.llm.Complete(ctx,
46        systemPrompt(language), string(payload))
47    if err != nil {
48        return "", fmt.Errorf(
49            "generating summary: %w", err)
50    }
51
52    // se il salvataggio fallisce segnaliamo l'errore

```

```

53 // ma restituiamo comunque il riepilogo
54 if err := s.store.Save(ctx, tenantID, geocID,
55     trigger, summary); err != nil {
56     fmt.Fprintln(os.Stderr, "saving summary:", err)
57 }
58 return summary, nil
59 }

```

Il ciclo `for` che raccoglie i dati è già pronto per l'estensione futura. Oggi raccoglie da una sola sorgente (i KPI), iterando sul risultato di `sourcesFor()`, ma domani basterà aggiungere una seconda fonte allo slice restituito senza modificare il resto della logica.

```

1 func (s *Service) sourcesFor(trigger string) []Source {
2     return []Source{s.kpi}
3 }

```

Il parametro `trigger` è oggi ignorato, perché tutti gli eventi attivano la stessa sorgente. È dichiarato ugualmente perché è la naturale sede della decisione futura: se un giorno l'apertura di un ticket dovesse consultare i documenti e il completamento di un'ispezione no, la differenza si esprimerebbe qui con una logica condizionale su `trigger`.

Le istruzioni per il modello risiedono nello stesso package. Il messaggio di sistema è fisso e ne varia soltanto la lingua, letta dalle impostazioni del tenant come richiede RQA-OB_02:

```

1 // internal/summary/prompt.go
2
3 const promptTemplate = `
4 You are a technical analyst. You receive (as JSON)
5 an asset with one or more data blocks: each block
6 has an "instructions" field explaining how to read
7 that data and a "result" field with the values.
8 Write a single summary, professional and in
9 {{LANGUAGE}}, following each block's "instructions"
10 and relying EXCLUSIVELY on the data provided.
11 [...]
12 `
13
14 func systemPrompt(language string) string {
15     return strings.ReplaceAll(promptTemplate,
16         "{{LANGUAGE}}", language)
17 }

```

La selezione della lingua avviene con `strings.ReplaceAll` anziché con la formattazione di stringhe del linguaggio, perché il testo delle istruzioni contiene caratteri come il segno di percentuale, che una funzione di formattazione interpreterebbe come segnaposto corrompendo il prompt

senza segnalarlo. Le regole omesse nell'estratto impongono al modello di attenersi ai soli dati presenti, omettere quanto manca invece di inventarlo, non riportare codici identificativi, esplicitare sempre le date e rispettare il limite di lunghezza richiesto da RQA-DE_02.

4.5.2 **kpi** — L'estrazione dei dati

Il package **kpi** realizza l'interfaccia **Source** ed è responsabile di trasformare un evento in un insieme di blocchi di dati. Quali dati compongano il riepilogo non è scritto nel codice: ogni tenant possiede nel proprio database una collection **kpi_query**, in cui ciascun documento descrive un blocco. Ecco un esempio:

```
1  {
2    "name": "ticket_stats",
3    "triggers": ["ticket_opened"],
4    "instructions": "Report the total number of
5      tickets of the asset and how many are still
6      open. Report the average first-response
7      time and [...]",
8    "query": {
9      "measures": [
10        "tickets_base.total_tickets",
11        "tickets_base.open_tickets",
12        "tickets_base.avg_response_days",
13        "tickets_base.avg_resolution_days"
14      ],
15      "dimensions": ["tickets_base.geoc_id"],
16      "filters": [{
17        "member": "tickets_base.geoc_id",
18        "operator": "equals",
19        "values": [{"GEOC_ID"}]
20      }]
21    }
22  }
```

Il campo **triggers** elenca gli eventi per i quali il blocco è pertinente, ed è ciò che realizza la differenziazione richiesta da RF-OB_03; il campo **instructions** accompagna i valori fino al modello e gli dice come leggerli; il campo **query** è l'interrogazione per il semantic layer, dove il segnaposto **{{GEOC_ID}}** prende il posto dell'entità. **geoc_id** è il nome con cui la piattaforma identifica l'oggetto a cui il riepilogo si riferisce, e il servizio lo riceve nel messaggio che gli chiede la produzione. Selezionare i blocchi da comporre è quindi un'unica interrogazione su MongoDB, con un filtro:

```
1 // internal/kpi/kpi.go
```

```

2
3 // match dei documenti il cui array contiene il trigger della richiesta
4 filter := bson.M{"triggers": trigger}

```

Ne discende la proprietà espressa da RQA-DE_01: aggiungere un blocco a un riepilogo, modificarne il taglio espositivo o abilitare un nuovo tipo di entità sono operazioni di configurazione, senza interventi sul codice. Il riepilogo dei ticket, aggiunto dopo quello delle ispezioni, è stato configurato senza toccare una riga di Go.

Le interrogazioni selezionate sono indipendenti tra loro e vengono eseguite in parallelo sfruttando le goroutine:

```

1 // internal/kpi/kpi.go Go
2
3 func (s *Service) runQueries(ctx context.Context,
4     tenantID, geocID, timezone string,
5     queries []QueryDoc) (map[string]any, error) {
6
7     blocks := map[string]any{}
8     var mu sync.Mutex
9     var wg sync.WaitGroup
10    var firstError error
11
12    for _, q := range queries {
13        if timezone != "" {
14            q.Query["timezone"] = timezone
15        }
16
17        wg.Add(1)
18        go func(q QueryDoc) {
19            defer wg.Done()
20
21            raw, err := json.Marshal(q.Query)
22            ...
23            queryJSON := strings.ReplaceAll(
24                string(raw), "{{GEOC_ID}}", geocID)
25
26            result, err := s.cube.Query(
27                ctx, tenantID, queryJSON)
28
29            mu.Lock()
30            defer mu.Unlock()
31            if err != nil {
32                if firstError == nil {
33                    firstError = fmt.Errorf(
34                        "query %s: %w", q.Name, err)

```

```

35     }
36     return
37 }
38 blocks[q.Name] = map[string]any{
39     "instructions": q.Instructions,
40     "result":      result,
41 }
42 }(q)
43 }
44 wg.Wait()
45 if firstError != nil {
46     return nil, fmt.Errorf(
47         "querying cube: %w", firstError)
48 }
49 return blocks, nil
50 }

```

La latenza complessiva del recupero è quella dell'interrogazione più lenta e non la somma di tutte, cosa che conta perché ogni riepilogo ne comprende più d'una: nella configurazione allestita durante lo stage sono tre sia per le ispezioni sia per i ticket. La mappa dei risultati è protetta da un mutex perché le goroutine vi scrivono in concorrenza, e viene trattenuto il primo errore: se una qualunque interrogazione fallisce, l'intera raccolta fallisce, coerentemente con il criterio esposto nella Sezione 4.3.

Il frammento mostra anche il trattamento del fuso orario richiesto da RQA-OB_03. Cube calcola i periodi e le granularità temporali in UTC quando non gli viene indicato diversamente: un'ispezione registrata alle prime ore del mattino in Italia ricadrebbe nel giorno precedente, e il riepilogo riporterebbe una data diversa da quella che l'operatore legge sulla scheda. Il fuso configurato dal tenant viene quindi aggiunto a ogni interrogazione; se il tenant non lo imposta, il campo non compare e resta il comportamento predefinito di Cube.

4.5.3 store — L'archivio dei riepiloghi

Il package `store` gestisce la collection `ai_summaries` presente nel database di ciascun tenant, dove ogni riepilogo è identificato dalla coppia entità-evento, e offre le tre operazioni corrispondenti ai tre momenti della sua vita.

La lettura distingue l'assenza dall'errore, restituendo un valore che dice se il riepilogo è stato trovato: per questo servizio l'assenza è una risposta e non un guasto. Il salvataggio è un *upsert*, quindi aggiorna il documento se esiste e lo crea altrimenti, appoggiandosi a un indice unico su entità ed evento che esclude i duplicati anche se due produzioni della stessa sintesi si concludessero insieme.

La cancellazione è l'unica delle tre che merita di essere vista, perché contiene al suo interno una scelta descritta nella Sezione 4.3:

```
1 // internal/store/store.go  Go
2
3 // cancella tutte le sintesi dell'entità a prescindere dal trigger
4 func (s *Store) Delete(ctx context.Context,
5     tenantID, geocID string) (int64, error) {
6     ...
7     res, err := s.summaries(tenantID).
8         DeleteMany(ctx, bson.M{"geoc_id": geocID})
9     if err != nil {
10         return 0, fmt.Errorf("delete: %w", err)
11     }
12
13     return res.DeletedCount, nil
14 }
```

Il filtro contiene il solo `geoc_id`, quindi la cancellazione riguarda i riepiloghi di tutti gli eventi dell'entità. Il conteggio restituito può essere zero senza che questo costituisca un errore: l'entità poteva non avere alcun riepilogo, ed è il caso normale quando la stessa segnalazione arriva due volte.

4.5.4 tenant — Le impostazioni del cliente

Il package `tenant` legge dalla collection `mytenant` le due impostazioni che condizionano il riepilogo, la lingua e il fuso orario.

Quella collection replica una collection omonima che la piattaforma possiede già nel proprio database, e da cui i valori sono stati copiati. La prima soluzione tentata era infatti leggerli direttamente da lì, ma avrebbe richiesto al servizio una seconda connessione verso un altro database, con le credenziali e la configurazione che ne conseguono. Trattandosi di due valori che cambiano di rado, si è preferito replicarli accanto alle altre collection di configurazione, rimandando al momento dell'integrazione in produzione la scelta di attingere alla sorgente originale.

Questa provenienza spiega due dettagli del codice che altrimenti sembrerebbero superflui. La ricerca filtra sul campo della lingua invece di prendere il primo documento disponibile, perché la collection della piattaforma distribuisce le proprie informazioni su più documenti e solo uno contiene le impostazioni cercate. E la sigla della lingua viene tradotta nel nome esteso da una mappa interna, invece di essere già scritta per esteso nei dati, perché è nella forma abbreviata che la piattaforma la registra. Entrambe le scelte servono a far sì che, il giorno in cui la sorgente tornasse a essere la collection originale, sia sufficiente cambiare il database

interrogato. Qualunque sigla non riconosciuta ricade sull'inglese, così che un dato mancante o imprevisto non blocchi mai la produzione.

```
1 // internal/tenant/tenant.go
2
3 var languages = map[string]string{
4     "it": "Italian",
5     "en": "English",
6     "es": "Spanish",
7     "de": "German",
8     "pt": "Portuguese",
9     "sk": "Slovak",
10    "zh": "Chinese",
11 }
12
13 const defaultLanguage = "English"
14
15 func (r *Reader) Get(ctx context.Context,
16     tenantID string) (language, timezone string, err error) {
17     ...
18     var d settingsDoc
19     err = r.db.Database(tenantID).
20         Collection("mytenant").
21         FindOne(ctx, bson.M{
22             "sync_locale": bson.M{"$exists": true},
23         }).
24         Decode(&d)
25     if errors.Is(err, mongo.ErrNoDocuments) {
26         return defaultLanguage, "", nil
27     }
28     ...
29     language, ok := languages[d.Locale]
30     if !ok {
31         language = defaultLanguage
32     }
33     return language, d.Timezone, nil
34 }
```

4.5.5 clients — Gli adattatori verso i sistemi esterni

Il package `clients` raccoglie gli adattatori verso i quattro sistemi con cui il servizio dialoga: Cube, il modello linguistico, MongoDB e AWS. Nessuno contiene logica di dominio, perché il loro compito è tradurre le chiamate del servizio nelle API dei rispettivi sistemi. Ciascuno però risolve un problema che il servizio non deve conoscere, e sono quattro problemi diversi.

Cube pretende un token a ogni interrogazione. Il client lo firma con l'identificativo del tenant e una breve scadenza; che cosa Cube ne faccia una volta ricevuto si vede nella Sezione 4.2.

```
1 // internal/clients/cube/cube.go
2
3 func (c *Client) token(tenantID string) (string, error) {
4     if tenantID == "" {
5         return "", fmt.Errorf("tenant_id is required")
6     }
7     now := time.Now()
8     claims := jwt.MapClaims{
9         "tenant_id": tenantID,
10        "iat":      now.Unix(),
11        "exp": now.Add(5 * time.Minute).Unix(),
12    }
13    return jwt.NewWithClaims(jwt.SigningMethodHS256,
14        claims).SignedString([]byte(c.secret))
15 }
```

Lo stesso client si fa carico di una particolarità del protocollo. Quando un'interrogazione richiede un calcolo non ancora in cache, Cube non risponde con il risultato ma con l'indicazione **Continue wait**, che invita a ripresentare la richiesta. Il client la ripresenta a intervalli di un secondo, entro un tempo massimo complessivo oltre il quale rinuncia anziché restare appeso.

Il modello linguistico si interroga con un'API pensata per le conversazioni: si invia un elenco di messaggi, ciascuno con un ruolo di riferimento. In questo caso il servizio ne utilizza soltanto due, uno di sistema con le istruzioni e uno utente con i dati, perché ogni riepilogo è una richiesta a sé e non esiste niente di precedente da ricordare. La temperatura è impostata a 0,2 perché a parità di dati il testo resti il più possibile stabile. Il codice è quello riportato nella Sezione 4.4.3.

MongoDB viene contattato una volta sola all'avvio, con un ping che verifica la connessione appena aperta. Un database irraggiungibile si scopre così quando il servizio parte, e non alla prima richiesta che lo interroga.

AWS è il client che costruisce la sessione con cui i due consumatori parlano alle code. In produzione non c'è niente da indicare: l'SDK ricava l'indirizzo del servizio dalla regione e usa le credenziali della macchina su cui gira. In sviluppo le code sono invece quelle di ElasticMQ, un container in esecuzione sulla macchina locale, e la configurazione ne riporta l'indirizzo perché il client lo usi al posto di quello di Amazon. Servono comunque delle credenziali, dato che l'SDK si rifiuta di partire senza, e siccome ElasticMQ non le controlla il client ne passa due inventate.

Il resto del servizio non si accorge della differenza: il codice è lo stesso, cambia una riga nel file di configurazione.

4.5.6 main.go — Il punto di ingresso e i consumatori

Il `main.go` è l'unico punto del programma che conosce tutte le parti appena viste. Il suo compito è costruirle nell'ordine giusto e collegarle: è qui che i tipi concreti vengono assegnati alle interfacce viste nella Sezione 4.5.1.

```
1 // cmd/context-service/main.go
2
3 func main() {
4     cfg, err := config.Load()
5     ...
6     ctx, stop := signal.NotifyContext(
7         context.Background(),
8         os.Interrupt, syscall.SIGTERM)
9     defer stop()
10
11     db, err := mongoclient.Connect(ctx, cfg.MongoURI)
12     ...
13     cubeClient := cube.New(cfg.CubeAPIURL, cfg.CubeSecret)
14     llm := openaiclient.New(cfg.OpenAIKey,
15         cfg.OpenAIURL, cfg.OpenAIModel)
16
17     summaries := store.New(db)
18     tenants := tenant.New(db)
19     kpiService := kpi.New(db, cubeClient)
20     summaryService := summary.New(
21         summaries, llm, kpiService, tenants)
22
23     var wg sync.WaitGroup
24
25     sess := awsclient.NewSession(
26         cfg.AWSRegion, cfg.SQSEndpoint)
27
28     producer := &sqsqconsumer.SQSconsumer{}
29     producer.SetSession(sess)
30     producer.EnsureExec(cfg.SQSQueueURL, 10, 3,
31         newProduceProcessor(summaryService), ctx, &wg)
32
33     invalidator := &sqsqconsumer.SQSconsumer{}
34     invalidator.SetSession(sess)
35     invalidator.EnsureExec(cfg.SQSInvalidateQueueURL,
36         10, 3, newInvalidateProcessor(summaries),
```

```

37     ctx, &wg)
38
39     startHTTP(ctx, cfg.HTTPAddr, summaries, &wg, stop)
40
41     wg.Wait() // aspetta la fine di tutte le goroutine
42 }

```

Tre aspetti di questo file stabiliscono il comportamento dell'intero servizio.

Il primo è il contesto. `signal.NotifyContext` produce un contesto che viene cancellato all'arrivo di una richiesta di arresto, e quel contesto viene passato a ogni ascoltatore: la cancellazione è il segnale unico con cui tutte le parti del servizio apprendono che è ora di fermarsi. È il meccanismo su cui si regge l'arresto controllato richiesto da RQA-OB_05: i consumatori smettono di prelevare messaggi nuovi e completano quelli in corso, i messaggi non ancora presi in carico restano sulla coda e non verranno elaborati.

Il secondo è il `WaitGroup`, il contatore con cui il programma principale attende la terminazione di tutte le goroutine prima di uscire. Senza questa attesa, all'uscita del `main` le elaborazioni in corso verrebbero interrotte a metà, vanificando l'arresto controllato appena descritto.

Il terzo è il consumo delle code, affidato alla libreria interna dell'azienda, uno dei vincoli del progetto (RV-OB_03). La libreria si fa carico dell'intero dialogo con SQS: la ricezione tramite *long polling* a blocchi di dieci messaggi, l'elaborazione in parallelo da parte di tre *worker*, la cancellazione dalla coda dei soli messaggi elaborati con successo e l'arresto alla cancellazione del contesto. Al servizio resta da fornire una sola cosa: la funzione che elabora il singolo messaggio.

Le funzioni di elaborazione delle due code, la produzione di un riepilogo e l'invalidazione, seguono lo stesso schema: sono costruite da una *factory* che riceve il componente da usare e restituisce la funzione da consegnare alla libreria di consumo.

```

1  // cmd/context-service/sqs_produce.go
2
3  // il tenant_id arriva negli attributi
4  type produceBody struct {
5      GeocID string `json:"geoc_id"`
6      Trigger string `json:"trigger"`
7  }
8
9  func newProduceProcessor(
10     svc *summary.Service) sqsconsumer.SQSMessageProcessor {
11
12     return func(m *sqs.Message) error {

```

```

13     attr, ok := m.MessageAttributes["tenant_id"]
14     if !ok || attr.StringValue == nil {
15         return fmt.Errorf("missing tenant_id attribute")
16     }
17     tenantID := *attr.StringValue
18
19     if m.Body == nil {
20         return fmt.Errorf("empty message body")
21     }
22     var body produceBody
23     err := json.Unmarshal([]byte(*m.Body), &body)
24     if err != nil {
25         return fmt.Errorf("invalid body: %w", err)
26     }
27     if tenantID == "" || body.GeocID == "" ||
28        body.Trigger == "" {
29         return fmt.Errorf(
30             "tenant_id, geoc_id and trigger are required")
31     }
32
33     aiSummary, err := svc.Produce(
34         context.Background(),
35         tenantID, body.GeocID, body.Trigger)
36     if err != nil {
37         return err
38     }
39     ...
40     return nil
41 }
42 }

```

La validazione precede ogni elaborazione, come previsto dalla Sezione 4.3: un messaggio privo dei dati necessari viene rifiutato prima che il flusso parta, senza spendere interrogazioni né chiamate al modello. L'errore restituito dalla funzione è anche il canale con cui si governa il destino del messaggio: se la funzione ritorna un errore la libreria non cancella il messaggio dalla coda, che torna disponibile per un nuovo tentativo.

Il consumatore dell'invalidazione ha la stessa forma, con la sola differenza che dopo la validazione chiama la cancellazione del riepilogo a database invece che della produzione.

4.5.7 http.go — La consultazione

L'ultimo ingresso del servizio è l'endpoint HTTP con cui la piattaforma consulta i riepiloghi. La sua caratteristica strutturale è ciò che non può

fare: il gestore della richiesta riceve un'interfaccia che espone la sola lettura, quindi la consultazione non ha accesso né alla produzione né alla cancellazione.

```
1 // cmd/context-service/http.go
2
3 type summaryReader interface {
4     Get(ctx context.Context, tenantID, geocID,
5         trigger string) (string, bool, error)
6 }
7
8 // GET /summary?tenant_id=...&geoc_id=...&trigger=...
9 func handleSummary(
10     summaries summaryReader) http.HandlerFunc {
11
12     return func(w http.ResponseWriter, r *http.Request) {
13         q := r.URL.Query()
14         tenantID := q.Get("tenant_id")
15         geocID := q.Get("geoc_id")
16         trigger := q.Get("trigger")
17         if tenantID == "" || geocID == "" || trigger == "" {
18             writeJSON(w, http.StatusBadRequest, summaryOut{
19                 Status: "error",
20                 ErrorDetails: "tenant_id, geoc_id and " +
21                     "trigger are required",
22             })
23             return
24         }
25
26         aiSummary, found, err := summaries.Get(
27             r.Context(), tenantID, geocID, trigger)
28         if err != nil {
29             writeJSON(w, http.StatusInternalServerError,
30                 summaryOut{
31                     Status: "error",
32                     ErrorDetails: err.Error(),
33                 })
34             return
35         }
36         if !found {
37             writeJSON(w, http.StatusOK,
38                 summaryOut{Status: "not_found"})
39             return
40         }
41         writeJSON(w, http.StatusOK,
42             summaryOut{Status: "ok", AiSummary: aiSummary})
```



```
43 }  
44 }
```

La risposta per il riepilogo assente ha stato HTTP 200 e non 404, con l'esito espresso dal campo **status** del corpo: l'assenza è una risposta valida prevista dal caso d'uso UC1, non un errore di chi ha chiesto, ed è la risposta su cui la piattaforma costruisce la proposta di produzione all'operatore.

L'arresto del server segue lo stesso segnale del resto del servizio, ma con un dettaglio che merita attenzione. Il contesto del servizio funziona qui da segnale di fermarsi: la goroutine resta in attesa sulla sua cancellazione, che arriva con la richiesta di arresto, ma il metodo **Shutdown** necessita di un contesto nuovo, che rappresenta il tempo massimo concesso alle richieste già in corso per terminare mentre il server smette di accettare nuove connessioni. Il contesto del servizio a questo punto è già cancellato, e passarlo a **Shutdown** lo farebbe rinunciare all'istante troncando le richieste a metà, cioè l'opposto dell'arresto controllato. Ne serve quindi uno nuovo, con dieci secondi propri, oltre i quali si chiude comunque perché un client fermo non trattienga il processo.

```
1 // cmd/context-service/http.go  
2  
3 wg.Add(1)  
4 go func() {  
5     defer wg.Done()  
6     <-ctx.Done() // aspetta ctrl+c o sigterm  
7  
8     shutdownCtx, cancel := context.WithTimeout(  
9         context.Background(), 10*time.Second)  
10    defer cancel()  
11    server.Shutdown(shutdownCtx)  
12 }()
```

Con questo l'esposizione dei componenti è completa: la produzione, l'archivio, le impostazioni, i client, i tre ingressi e il punto che li collega. Il modo in cui questo insieme è stato verificato, sia nelle parti sia nel flusso completo, è l'oggetto del Capitolo 5.

Capitolo 5

Verifica e validazione

In questo capitolo descrivo come ho verificato che il servizio funzioni correttamente e come ne ho validato il risultato con l'azienda, riportando il grado di soddisfacimento dei requisiti e i limiti delle prove svolte.

5.1 Approccio adottato

Le attività descritte in questo capitolo rispondono a due domande distinte:

- **verifica** chiede se il servizio faccia ciò che è stato specificato;
- **validazione** chiede se ciò che è stato specificato sia effettivamente utile a chi lo userà

Il servizio è stato verificato in due modi, ai quali sono dedicate le due sezioni seguenti. I test automatici provano le singole parti in isolamento, si eseguono con un solo comando e possono quindi essere ripetuti a ogni modifica. I test di sistema riguardano il servizio intero collegato ai sistemi veri, nell'ambiente di sviluppo locale che riproduce l'infrastruttura di produzione e attinge ai dati reali dell'ambiente di *staging* aziendale. Sono stati eseguiti a mano perché mettono in gioco situazioni che un test automatico non riproduce, come spegnere una sorgente dati o fermare il servizio mentre sta elaborando.

Va subito detto, che i test automatici non sono stati scritti insieme al codice ma nella parte conclusiva dello stage. Durante lo sviluppo la verifica è stata manuale, condotta eseguendo il flusso e osservandone il risultato. La scelta ha una ragione, cioè che la forma del servizio è cambiata a fondo più di una volta e i test scritti presto sarebbero stati riscritti altrettante, ma ha anche un costo che è giusto riconoscere: alcuni difetti sono stati individuati più tardi di quanto sarebbe accaduto altrimenti.

5.2 Test automatici

I test sono scritti con il package `testing` della libreria standard di Go, senza librerie esterne (coerentemente con la filosofia del linguaggio), e sono di due tipi:

- **Test di unità.** Riguardano i package in cui la logica è propria del servizio e le dipendenze sono dichiarate come interfacce, secondo quanto descritto nel Capitolo 4. Al posto del modello linguistico, del semantic layer e dell'archivio vengono fornite realizzazioni simulate, che restituiscono valori prestabiliti e registrano le chiamate ricevute. La parte in esame resta così isolata dal resto, e il flusso di produzione si verifica per intero senza chiamare il modello (senza costi, senza rete e con esito sempre uguale a parità di ingressi).
- **Test di integrazione.** Riguardano i package che parlano con MongoDB, dove ciò che va verificato è proprio l'interrogazione e non la logica che la circonda. Questi test avviano quindi un'istanza reale di MongoDB in un container, vi inseriscono i documenti necessari, eseguono le operazioni e distruggono il container al termine. L'avvio e la distruzione del container sono raccolti in una funzione di supporto, che ciascuno di questi test richiama con una riga.

```
1 // internal/mongotest/mongotest.go
2
3 // avvia un mongo in un container usa e getta e ne restituisce
4 // il client; container e connessione vengono chiusi a fine test
5 func Start(t *testing.T) *mongo.Client {
6     t.Helper()
7     ctx := context.Background()
8
9     container, err := mongodb.Run(ctx, "mongo:7")
10    if err != nil {
11        t.Fatalf("avvio del container: %v", err)
12    }
13    t.Cleanup(func() {
14        testcontainers.TerminateContainer(container)
15    })
16
17    uri, err := container.ConnectionString(ctx)
18    ...
19    client, err := mongo.Connect(ctx,
20        options.Client().ApplyURI(uri))
21    ...
22    t.Cleanup(func() { client.Disconnect(ctx) })
23
24    return client
```

Le due chiamate a **t.Cleanup** sono ciò che rende sostenibile questo approccio: la chiusura del container e della connessione viene registrata subito dopo l'apertura ed eseguita comunque, anche se il test fallisce a metà (senza di esse un test interrotto lascerebbe container attivi sulla macchina).

In tutto la suite comprende sedici casi, otto di unità e otto di integrazione. La Tabella 9 riassume che cosa verifica ciascun gruppo e a quali requisiti si riferisce.

Package	Comportamento verificato	Requisiti
store	Un riepilogo salvato viene riletto correttamente; un secondo salvataggio sulla stessa chiave lo aggiorna invece di duplicarlo; la ricerca su un'entità priva di riepiloghi non trova nulla; l'invalidazione cancella tutti i riepiloghi dell'entità e non solo quelli di un evento, e invalidare un'entità che non ne ha non produce errore.	RF-OB_06, RF-OB_07, RQA-OB_07
tenant	Le impostazioni del cliente vengono lette dal documento corretto anche in presenza di altri documenti nella stessa collection; una lingua configurata ma non supportata ricade sull'inglese; in assenza di impostazioni la lingua ricade sull'inglese e il fuso orario resta non specificato.	RQA-OB_02, RQA-OB_03
kpi	Nell'interrogazione inviata al semantic layer il segnaposto è sostituito con l'identificativo dell'entità; i risultati sono raccolti con le rispettive istruzioni di lettura; il fuso orario del cliente viene aggiunto all'interrogazione, e non compare quando non è impostato.	RF-OB_03, RQA-OB_03

Package	Comportamento verificato	Requisiti
summary	Se il riepilogo esiste già viene restituito senza chiamare il modello; se non esiste viene percorso l'intero flusso, dalla raccolta dei dati alla chiamata al modello al salvataggio.	RF-OB_01, RF-OB_05, RQA-OB_07
cmd	La consultazione restituisce il riepilogo quando esiste e ne segnala l'assenza quando non esiste, senza trattarla come un errore; una richiesta priva di un parametro obbligatorio viene respinta; un errore dell'archivio produce una risposta di errore e non l'interruzione del servizio.	RF-OB_04, RF-OB_06, RQA-OB_06

Tabella 9: Test automatici e requisiti verificati.

L'intera suite viene eseguita con un solo comando e termina con esito positivo. La copertura raggiunta è riportata nella Tabella 10.

Package	Copertura	Nota
tenant	92,3%	
store	87,0%	
summary	77,8%	Non coperti i rami di errore delle dipendenze
kpi	45,7%	Non coperta la lettura delle interrogazioni da MongoDB
cmd	19,8%	Coperta la sola consultazione; avvio, configurazione e funzioni di consumo delle code non sono sotto test
clients	0%	Adattatori verso i sistemi esterni, privi di logica propria

Tabella 10: Copertura delle istruzioni per package.

I valori vanno letti per quello che sono. La copertura misura quante istruzioni vengono eseguite durante i test, non quanti comportamenti significativi siano stati verificati, ed è quindi un indicatore utile a individuare le zone d'ombra più che a certificare la qualità.

5.3 Test di sistema manuali

I test automatici verificano le parti; resta da verificare che il servizio funzioni quando le parti sono collegate ai sistemi reali. Per questo è stato allestito un ambiente di sviluppo locale che riproduce l'infrastruttura di produzione: MongoDB, Cube e la coda vengono eseguiti come container, con ElasticMQ al posto di SQS. Il codice del servizio è identico a quello che andrebbe in produzione, e cambia soltanto l'indirizzo della coda nella configurazione. I dati sono quelli reali dell'ambiente di staging aziendale. Su questo ambiente ho eseguito a mano i seguenti scenari.

Produzione di un riepilogo inviato il comando sulla coda, il servizio recupera le interrogazioni del cliente, ottiene i valori dal semantic layer, produce il testo e lo conserva. Il testo è stato confrontato con i dati di partenza per accertare che ogni valore riportato vi trovasse riscontro.

Consultazione di un riepilogo esistente l'endpoint restituisce il testo conservato senza attivare alcuna produzione.

Consultazione di un riepilogo assente l'endpoint segnala l'assenza. È la risposta su cui si regge l'intero disegno su richiesta, perché è quella che permette alla piattaforma di proporre la produzione all'operatore.

Invalidazione inviata la segnalazione di modifica, i riepiloghi dell'entità vengono cancellati e la consultazione successiva li dà per assenti.

Ripetizione dell'invalidazione la stessa segnalazione inviata due volte cancella due riepiloghi al primo passaggio e nessuno al secondo, senza errore. È la verifica diretta di RQA-OB_07.

Indisponibilità di una sorgente spegnendo il semantic layer, la produzione fallisce senza conservare alcun riepilogo parziale e il messaggio resta sulla coda. La consultazione continua a rispondere per i riepiloghi già presenti.

Arresto durante l'elaborazione fermando il servizio mentre una produzione è in corso, questa viene portata a termine prima della chiusura, e i messaggi non ancora presi in carico restano sulla coda. L'ultimo scenario merita una precisazione: la mia prova ne accerta il comportamento nominale, ma non copre il caso in cui l'elaborazione

ecceda il limite di tempo previsto per l'arresto. Quel caso è affidato all'idempotenza, verificata separatamente.

5.4 Validazione del risultato

Il confronto con il tutor aziendale ha riguardato soprattutto le scelte di approccio e di realizzazione, quelle discusse nel Capitolo 4, ma ha toccato fin dall'inizio anche il contenuto del riepilogo. L'indicazione di partenza era di restare sui KPI aggregati, accompagnati da qualche analisi elementare, come l'andamento del rischio negli anni. I primi riepiloghi prodotti sui dati reali sono stati giudicati adeguati già in quella forma.

Di mia iniziativa avevo aggiunto al riepilogo delle ispezioni una quantità di informazioni puntuali sulle singole occorrenze. Il tutor ha chiesto di toglierle, per mantenere il riepilogo sul piano generale degli aggregati. Le informazioni puntuali sono rientrate più tardi con lo scenario dei ticket, e in quella forma sono state accolte: dei ticket aperti il riepilogo riporta quando sono stati aperti, la priorità, lo stato e di che cosa trattano.

Anche le due modifiche successive sono nate da una mia proposta, poi approvata dal tutor, e vengono dal confronto fra il riepilogo e la scheda che ha davanti l'operatore. Il testo veniva prodotto in inglese, inservibile per un operatore italiano sul campo, e le date calcolate in UTC dal semantic layer non coincidevano con quelle mostrate dalla piattaforma: uno scarto di un giorno su una data di chiusura non è un difetto che il codice segnali in alcun modo. Ne sono derivati i requisiti RQA-OB_02 e RQA-OB_03.

5.5 Grado di soddisfacimento dei requisiti

La Tabella 11 riporta l'esito per ciascun requisito individuato nel Capitolo 3.

Tipologia	Priorità	Individuati	Soddisfatti	Percentuale
Funzionali	Obbligatorie	8	8	100%
Funzionali	Desiderabili	1	0	0%

Tipologia	Priorità	Individuati	Soddisfatti	Percentuale
Funzionali	Opzionali	1	0	0%
Qualitativi	Obbligatori	7	7	100%
Qualitativi	Desiderabili	2	1	50%
Di vincolo	Obbligatori	3	3	100%
Di vincolo	Desiderabili	1	0	0%
Totale		23	19	83%

Tabella 11: Grado di soddisfacimento dei requisiti.

Tutti i requisiti obbligatori sono soddisfatti. I quattro non soddisfatti sono desiderabili o opzionali:

RF-DE_01 l'indicizzazione delle entità per il recupero da documenti non strutturati, corrispondente all'obiettivo O03, analizzata sul piano degli approcci ma non realizzata;

RF-OP_01 la risposta a richieste dell'utente in modalità agente, corrispondente all'obiettivo O04, che presuppone il precedente;

RV-DE_01 l'integrazione nell'architettura e nel ciclo di rilascio della piattaforma, corrispondente all'obiettivo desiderabile D01;

RQA-DE_02 il contenimento della sintesi entro una lunghezza massima, che ho tentato di ottenere e che la sezione seguente documenta.

5.5.1 Tracciamento delle verifiche

La Tabella 12 indica per ciascun requisito il modo in cui è stato verificato. I vincoli si verificano per ispezione, perché riguardano come il servizio è fatto e non come si comporta; i requisiti lasciati fuori dal perimetro non hanno verifica perché non hanno realizzazione.

Requisito	Verifica
RF-OB_01	Test di unità e prova sul flusso completo
RF-OB_02	Validazione del risultato con il tutor aziendale
RF-OB_03	Test di unità e prova sul flusso completo
RF-OB_04	Test di unità e prova sul flusso completo
RF-OB_05	Test di unità e prova sul flusso completo
RF-OB_06	Test di integrazione e prova sul flusso completo
RF-OB_07	Test di integrazione e prova sul flusso completo
RF-OB_08	Prova sul flusso completo: ogni comando arriva dalla coda
RF-DE_01	Non realizzato
RF-OP_01	Non realizzato
RQA-OB_01	Prova sul flusso completo: confronto fra i valori nel testo e i dati di partenza
RQA-OB_02	Test di integrazione e prova sul flusso completo
RQA-OB_03	Test di unità, test di integrazione e validazione del risultato
RQA-OB_04	Non verificato: la configurazione comprende un solo cliente
RQA-OB_05	Prova sul flusso completo: arresto durante l'elaborazione
RQA-OB_06	Test di unità e prova sul flusso completo: indisponibilità del semantic layer

Requisito	Verifica
RQA-OB_07	Test di unità, test di integrazione e ripetizione dell'invalidazione
RQA-DE_01	Verifica sul campo: il riepilogo dei ticket è stato aggiunto senza modifiche al codice
RQA-DE_02	Misura sperimentale su ventuno generazioni: requisito non soddisfatto
RV-OB_01	Ispezione del codice
RV-OB_02	Ispezione della configurazione
RV-OB_03	Ispezione del codice
RV-DE_01	Non realizzato

Tabella 12: Modo di verifica di ciascun requisito.

- Attenzione 1: RQA-DE_01 (configurabilità per cliente) non ha un test dedicato ma una verifica fatta «sul campo», infatti il secondo tipo di entità è stato messo in esercizio a servizio già funzionante, e per farlo sono bastati nuovi documenti di configurazione (oltre al setup del semantic layer).
- Attenzione 2: RQA-OB_04 (segregazione fra clienti) è l'unico requisito obbligatorio privo di verifica, per la ragione discussa più avanti fra i limiti.

5.5.2 Approfondimento: la lunghezza del riepilogo

RQA-DE_02 chiede che la sintesi si mantenga entro una lunghezza massima, scegliendo i contenuti in ordine di importanza quando i dati eccedono. Durante lo stage questo requisito è stato affrontato nel modo più semplice possibile: dichiarare il limite fra le istruzioni al modello. Facendo qualche test ho subito notato che il limite non veniva rispettato. Per capire se il limite fosse seguito almeno come indicazione, ho testato empiricamente il servizio. Ho generato sette volte lo stesso riepilogo per 3 limiti di lunghezza differenti - 500, 650 e 750 caratteri - lasciando invariati tutti gli altri parametri.

Limite	Le sette misure	Media	Entro
500	804, 844, 820, 920, 813, 727, 860	827	0/7
650	695, 794, 789, 822, 822, 689, 704	759	0/7
750	825, 793, 867, 808, 771, 777, 785	804	0/7

Tabella 13: Lunghezza in caratteri di sette generazioni per ciascun limite dichiarato nell'istruzione.

Il numero scritto nell'istruzione non governa la lunghezza del testo. Abbassare il limite a 500 ha prodotto i testi più lunghi dei tre gruppi; alzarlo a 750 non li ha allungati in proporzione. Nessuna delle ventuno generazioni è rientrata nel proprio limite.

A governare la lunghezza è il contenuto. L'entità usata per le prove ha un ticket aperto e cinque chiusi di recente, e le istruzioni dei singoli blocchi chiedono per ciascuno la data, la priorità, lo stato e una descrizione: il limite complessivo e le istruzioni di dettaglio sono richieste in conflitto, e il modello lo risolve a favore del dettaglio.

Il requisito non è quindi soddisfatto. Il Capitolo 6 riprende la questione suggerendo approcci differenti.

5.6 Limiti della verifica svolta

Per completezza dichiaro ciò che le prove descritte non coprono.

Il servizio non è stato provato sotto carico. Non esistono misure su quante richieste di produzione possa smaltire nell'unità di tempo, né sul comportamento della coda quando le richieste si accumulano. Il dato mancherebbe comunque di un riferimento realistico, dal momento che il servizio non è mai stato esposto a un traffico vero.

Il comportamento in caso di messaggi ripetutamente malformati non è stato provato, coerentemente con il fatto che la coda di scarto non è stata configurata: è la lacuna già dichiarata nel Capitolo 4.

Anche la segregazione dei dati fra clienti è rimasta senza prova. Il requisito RQA-OB_04 è soddisfatto per costruzione, perché come descritto nel Capitolo 4 ogni cliente ha un proprio database e un proprio modello compilato, ma la configurazione allestita durante lo stage comprende un solo cliente. Manca quindi l'unica prova che varrebbe davvero: un secondo cliente a cui i dati del primo risultino inaccessibili.

La validazione si è fermata al giudizio del tutor aziendale. Nessun operatore ha usato i riepiloghi nel proprio lavoro, quindi manca il riscontro

che conterebbe di più, cioè se servano davvero a chi deve decidere se intervenire su un impianto.

Infine, l'assenza dell'integrazione in produzione fa sì che tutte le prove siano state condotte in un ambiente che riproduce quello reale ma non lo è. La riproduzione è fedele nei componenti e nei dati, ma non nella scala, nella concorrenza tra più consumatori e nelle condizioni di rete.

Capitolo 6

Conclusioni

In questo capitolo valuto il raggiungimento degli obiettivi, indico le direzioni in cui il lavoro può proseguire, riassumo ciò che ho imparato e trago un bilancio personale dell'esperienza.

6.1 Raggiungimento degli obiettivi

Il piano di lavoro fissava quattro obiettivi obbligatori e uno desiderabile. La Tabella 14 ne riporta l'esito.

Codice	Obiettivo	Esito
O01	Ricezione di eventi asincroni dalle altre componenti della piattaforma, con caricamento delle configurazioni del cliente e delle entità interessate	Raggiunto
O02	Valutazione di KPI specifici per entità e cliente, e interfacciamento con l'IA generativa per la produzione della sintesi testuale	Raggiunto
O03	Indicizzazione delle entità per il recupero di documenti non strutturati	Non raggiunto
O04	Ricezione di richieste utente in modalità agente, con recupero documentale e risposta	Non raggiunto
D01	Integrazione nell'architettura e nel ciclo di rilascio della piattaforma	Non raggiunto

Tabella 14: Raggiungimento degli obiettivi del piano di lavoro.

Gli obiettivi raggiunti sono quelli che riguardano i dati strutturati, e su di essi il lavoro è arrivato più lontano di quanto il piano richiedesse: non solo il servizio produce le sintesi, ma le conserva, le mantiene allineate ai dati e le rende consultabili. Al servizio si affianca il modello dati del semantic layer, dove le metriche sono state definite e da cui i valori provengono, più una suite di test automatici e un ambiente di sviluppo che riproduce l'infrastruttura con un comando.

Gli obiettivi O03 e O04 riguardano invece i dati non strutturati e non sono stati realizzati. La decisione, presa con il tutor aziendale a metà stage, è discussa nel Capitolo 1 e le sue ragioni tecniche nel Capitolo 4: l'estensione ai documenti richiedeva un'infrastruttura autonoma di indicizzazione e recupero, il cui sviluppo avrebbe occupato tutto il tempo restante lasciando incompiuto anche il flusso sui dati strutturati. Fra due fronti aperti a metà e uno chiuso, si è preferito il secondo.

L'obiettivo desiderabile D01 non è stato raggiunto per ragioni di tempo e di coordinamento con il ciclo di rilascio interno, e nella settimana conclusiva l'azienda ha preferito destinare il tempo residuo a un affiancamento formativo con uno sviluppatore del team.

Il risultato complessivo è quindi un servizio completo e verificato sul perimetro dei dati strutturati, configurato per un cliente e per due dei tre tipi di entità della piattaforma, e uno studio documentato ma non realizzato sul perimetro restante. Ciò che manca perché entri in esercizio è elencato nella sezione seguente.

6.2 Sviluppi futuri

Il lavoro può proseguire in più direzioni, che elenco dalla più immediata alla più impegnativa.

Configurazione del terzo tipo di entità. Gli asset sono rimasti fuori dalla configurazione allestita durante lo stage. Come osservato nel Capitolo 4, colmare la lacuna non richiede sviluppo: il servizio non distingue i tipi di entità, e il terzo percorre le stesse istruzioni dei primi due. Occorrono la definizione delle metriche nel semantic layer e la configurazione delle interrogazioni corrispondenti.

Coda di scarto. Un messaggio malformato torna oggi sulla coda indefinitamente. Configurare una *dead letter queue* dopo un numero prestabilito di tentativi lo toglie dal ciclo e lo conserva per l'ispezione. È una modifica di configurazione dell'infrastruttura, non del servizio.

Contenimento della lunghezza. È il requisito RQA-DE_02, rimasto insoddisfatto. La sua importanza cresce con la mole di dati: l'entità usata per le prove ha sei ticket, ma un'ispezione con dieci anni di storia produrrebbe un testo in cui persino gli indicatori di base rischiano di

non trovare posto. Serve che il servizio, davanti a più dati di quanti ne possa esporre, scelga.

L'esperimento del Capitolo 5 indica dove non intervenire: il numero dichiarato nell'istruzione non ha effetto, perché a determinare la lunghezza è quanto c'è da dire. La leva sta quindi a monte. Ridurre gli elementi passati al modello, per esempio abbassando il numero di ticket recuperati da ciascuna interrogazione, accorcia il testo in modo deterministico ed è pura configurazione. Spostare il vincolo dal testo complessivo al singolo blocco, chiedendo una frase per ticket anziché un totale di caratteri, parla al modello nella stessa forma delle istruzioni che già esegue con fedeltà. A valle, un controllo che misuri il testo e ne chieda una riscrittura più breve quando eccede fornirebbe la garanzia, al prezzo di una seconda chiamata.

Solo a quel punto avrebbe senso indicare nelle istruzioni un ordine di importanza fra i blocchi: quando esiste un budget effettivo, quell'ordine diventa il criterio con cui decidere che cosa sacrificare, mentre oggi il modello non arriva mai a doversi porre il problema.

Integrazione in produzione. Corrisponde all'obiettivo D01 e comprende la definizione del topic da cui provengono le segnalazioni di modifica, la creazione delle code dedicate, l'inserimento del servizio nel ciclo di rilascio e la strumentazione per l'osservabilità, oggi limitata alla registrazione degli eventi sullo standard error.

Generalizzazione dell'accesso ai dati. L'interfacciamento tra MongoDB e il semantic layer si appoggia oggi a una vista materializzata su PostgreSQL, che va scritta per ciascun cliente. Fra le alternative esaminate nel Capitolo 4, l'esportazione in formato Parquet interrogata da DuckDB è quella che le eliminerebbe senza aggiungere componenti all'infrastruttura, dal momento che DuckDB è già incluso in Cube.

Estensione ai dati non strutturati. È l'obiettivo O03, e con esso l'O04 che lo presuppone. Lo studio svolto indica come soluzione appropriata un recupero ibrido, che combina ricerca vettoriale e ricerca lessicale con una fase di re-ranking. La struttura del servizio è predisposta ad accoglierlo: la produzione raccoglie i blocchi di contesto da un insieme di sorgenti, e una sorgente documentale si aggiunge alle esistenti senza modificare il flusso. Anche la modalità agente di O04 può essere affrontata restando in Go, dove sono ormai disponibili framework dedicati alla costruzione di sistemi multi-agente [7].

6.3 Conoscenze acquisite

Sono entrato in azienda senza conoscere il linguaggio Go, che ho imparato durante lo stage e nel quale ho scritto l'intero servizio. Al di là della sintassi, che si assimila in fretta, quello che ho dovuto acquisire è il modo

in cui in Go si organizza un programma: le interfacce dichiarate da chi le usa e non da chi le fornisce, la gestione esplicita degli errori a ogni chiamata, la concorrenza come strumento ordinario e non come tecnica avanzata. Sono convenzioni che all'inizio sembrano arbitrarie e di cui si capisce il senso solo quando il programma cresce.

Ho imparato a lavorare con i modelli linguistici, e soprattutto a diffidarne: la loro affidabilità dipende dal compito assegnato. Buona parte della progettazione descritta in questa relazione consiste nel disegnare il sistema attorno a questa distinzione.

Ho conosciuto il semantic layer, che non avevo mai incontrato negli studi, e ne ho scritto i modelli. Definire una metrica si è rivelato meno meccanico di quanto pensassi.

Ho lavorato con la comunicazione asincrona a code, e ho capito sul campo perché l'idempotenza non sia una raffinatezza teorica ma la conseguenza diretta di una garanzia di consegna.

Sul piano non tecnico, l'acquisizione più utile riguarda i requisiti. Solitamente in ambito accademico arrivano già scritti; in azienda vanno ricavati, perché stanno nella testa delle persone e vanno distinti dalle soluzioni che ha già in mente chi li esprime. Alcuni però non stanno nella testa di nessuno, e compaiono soltanto quando qualcuno guarda un vero risultato.

6.4 Valutazione personale

Glossario

allucinazione: Affermazione prodotta da un modello linguistico che risulta plausibile e ben formulata ma non corrisponde ad alcun dato reale. È la conseguenza del fatto che il modello ottimizza la plausibilità del testo e non la sua verità.

BM25 – Best Matching 25: Funzione di ordinamento usata nella ricerca testuale, che valuta la pertinenza di un documento rispetto a una interrogazione sulla base della frequenza dei termini. A differenza della ricerca vettoriale opera sulle parole effettive, ed è quindi efficace sulle corrispondenze esatte come codici e sigle.

CI – Continuous integration: Pratica di programmazione che prevede la frequente integrazione del codice prodotto verso il ramo principale del repository Git. Generalmente prima di poter fare questa integrazione il sistema esegue dei test automatici (compilazione, unità, ecc...).

DLQ – Dead letter queue: Coda di scarto sulla quale un sistema di messaggistica sposta i messaggi la cui elaborazione è fallita per un numero prestabilito di volte, così da toglierli dal ciclo dei tentativi e conservarli per l'ispezione.

goroutine: Unità di esecuzione concorrente del linguaggio Go, gestita dal runtime del linguaggio anziché dal sistema operativo, e per questo molto più leggera di un thread.

idempotenza: Proprietà di un'operazione che, ripetuta più volte con gli stessi argomenti, produce lo stesso effetto di una singola esecuzione. È necessaria quando l'infrastruttura può consegnare due volte lo stesso messaggio.

JWT – JSON Web Token: Formato standard di token che racchiude un insieme di dichiarazioni firmate digitalmente. Chi lo riceve può verificarne l'autenticità senza consultare l'emittente, e usarne il contenuto per stabilire identità e permessi di chi presenta il token.

KPI – Key Performance Indicator: Indicatore quantitativo che misura una grandezza rilevante per il funzionamento di un'organizzazione o di un processo. In questa relazione i KPI sono i valori aggregati calcolati sulle entità della piattaforma, come il numero di ticket aperti o il tempo medio di risoluzione.

LLM – Large Language Model: Modello di apprendimento automatico addestrato su grandi quantità di testo, capace di comprendere

e produrre linguaggio naturale. Genera il testo più probabile date le istruzioni ricevute, senza alcuna verifica rispetto a una fonte.

multi-tenancy: Modello architetturale in cui una sola installazione di un'applicazione serve più organizzazioni clienti, dette tenant, mantenendone i dati separati e non accessibili le une alle altre.

Parquet: Formato di file per la memorizzazione di dati organizzati per colonne anziché per righe, pensato per le interrogazioni analitiche, che leggono poche colonne su molte righe.

RAG – Retrieval-Augmented Generation: Approccio in cui, prima di interrogare un modello linguistico, si recuperano da una base documentale i frammenti pertinenti alla domanda e li si forniscono al modello insieme ad essa, così che la risposta si fondi su documenti reali.

re-ranking: Fase successiva a un recupero documentale, in cui i risultati ottenuti vengono riordinati da un secondo modello, più accurato e più costoso del primo, applicato al solo insieme ristretto dei candidati.

SaaS – Software as a Service: Modello di distribuzione del software in cui l'applicazione è eseguita e mantenuta dal fornitore e resa disponibile ai clienti attraverso la rete, senza che questi debbano installarla o gestirne l'infrastruttura.

semantic layer: Strato intermedio fra le sorgenti dati e le applicazioni, nel quale le metriche di business sono definite una sola volta e in modo dichiarativo. Le applicazioni le interrogano attraverso un'API, ottenendo valori coerenti fra loro.

SNS – Amazon Simple Notification Service: Servizio di distribuzione di messaggi di AWS organizzato per argomenti (topic). Un messaggio pubblicato su un topic viene recapitato in copia a tutti i sottoscrittori, permettendo a più consumatori di ricevere lo stesso evento.

SQS – Amazon Simple Queue Service: Servizio di code gestite di AWS. Un messaggio inserito in coda viene consegnato a un solo consumatore e rimosso soltanto dopo che questi ne ha dichiarato la corretta elaborazione, con garanzia di consegna almeno una volta.

staging: Ambiente che replica quello di produzione e ne utilizza dati realistici, impiegato per le prove che precedono il rilascio senza incidere sul servizio effettivamente in uso.

testcontainers: Libreria che permette a un test automatico di avviare un servizio reale (per esempio un database) dentro un container, utilizzarlo e distruggerlo al termine, così da verificare il codice contro il sistema vero anziché contro una sua imitazione.

Text-to-SQL: Approccio in cui un modello linguistico traduce una domanda espressa in linguaggio naturale nell'interrogazione SQL corrispondente, che viene poi eseguita sul database.

Bibliografia

- [1] R. Channe e S. Mirchandani, «Metric Semantic Layer: How Lyft Governs and Scales Key Data Definitions», *Lyft Engineering*, giu. 2026, [Online]. Disponibile su: <https://eng.lyft.com/metric-semantic-layer-how-lyft-governs-and-scales-key-data-definitions-56bee3643c29>
- [2] P. Bhardwaj, «Hybrid Search and Re-Ranking in Production RAG», *Towards Data Science*, mag. 2026, [Online]. Disponibile su: <https://towardsdatascience.com/hybrid-search-and-re-ranking-in-production-rag/>
- [3] «Extending cubes». [Online]. Disponibile su: <https://docs.cube.dev/docs/data-modeling/extending-cubes>
- [4] G. Hohpe e B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [5] A. Cockburn, «Hexagonal Architecture». [Online]. Disponibile su: <https://alistair.cockburn.us/hexagonal-architecture/>
- [6] E. Gamma, R. Helm, R. Johnson, e J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [7] T. Klopfenstein e S. K. Maddula, «Build reliable multi-agent applications with ADK Go 2.0». [Online]. Disponibile su: <https://developers.googleblog.com/announcing-adk-go-20/>